上海科技大学
ShanghaiTech University

CS177 Bioinformatics: Software Development and Applications

Lecture 8: Introduction to Deep Learning

Part 3. Graph Neural Networks and Knowledge Graphs

Jie Zheng (郑杰)

PhD, Associate Professor

School of Information Science and Technology (SIST), ShanghaiTech University

Spring, 2025

立志成才 报国裕民

Learning objectives

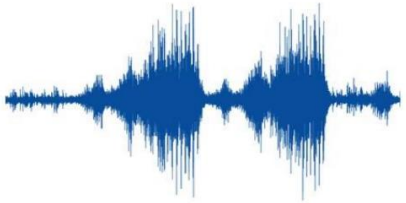
- After taking this lecture, you should be able to:
  - Point out the motivation for graph neural networks (GNNs)
  - Explain the main idea of graph convolutional networks (GCNs)
  - In regard to limitations of GCNs, describe a few extensions of GCNs (e.g. GNN with attention)
  - List and explain three main categories of applications of GNNs (node classification, graph classification, link prediction)
  - Define what is a knowledge graph (KG)
  - Describe how to construct and complete a KG

Graph-Structured Data

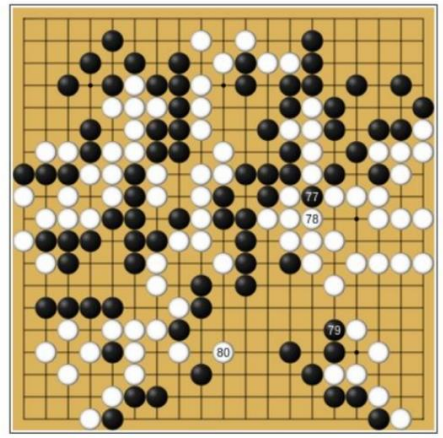
The success story of deep learning

IMAGENET

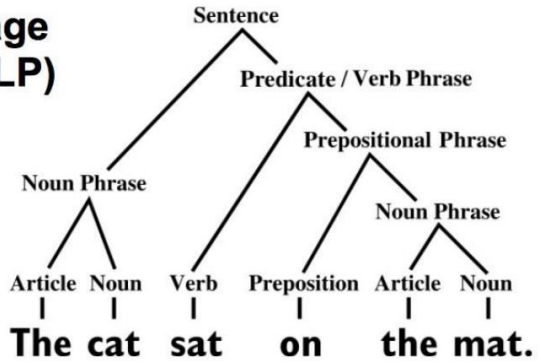
Speech data



Grid games



Natural language processing (NLP)



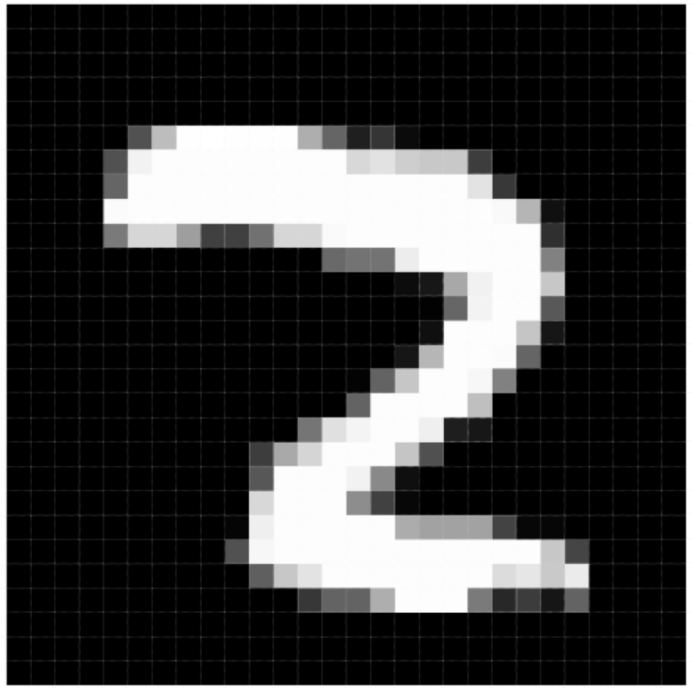
...

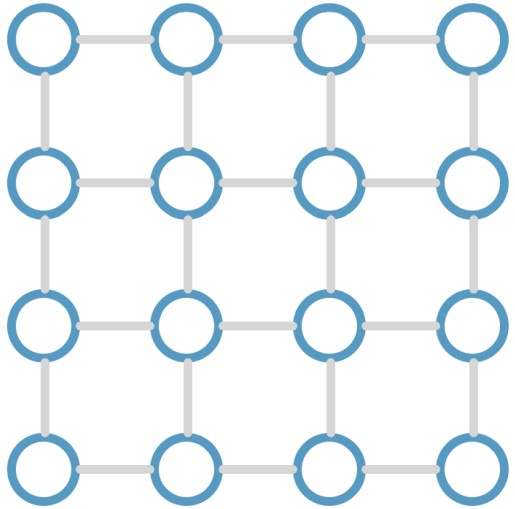
Deep neural nets that exploit:

- translation equivariance (weight sharing)
- hierarchical compositionality

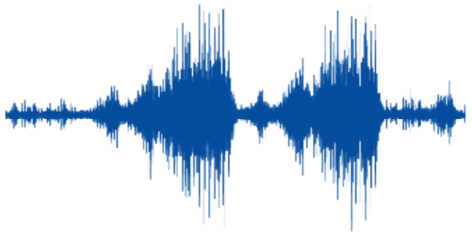
Recap: Deep learning on Euclidean data

Euclidean data: grids, sequences…





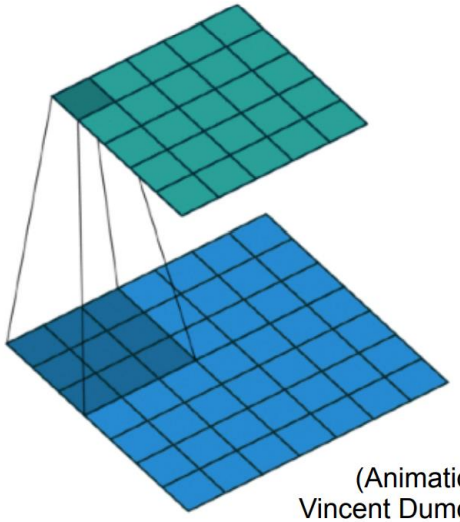
2D grid



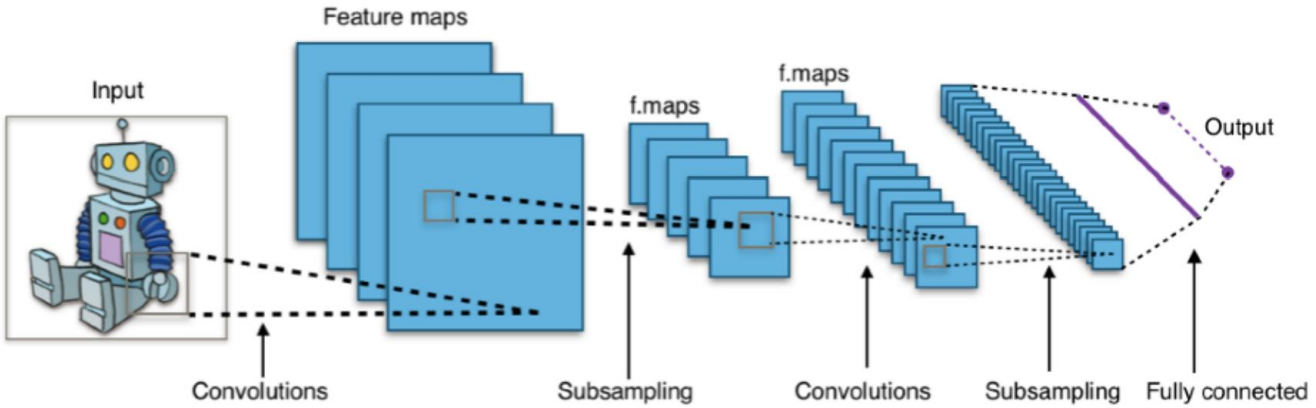
1D grid

Recap: Deep learning on Euclidean data

Convolutional neural networks (CNNs)

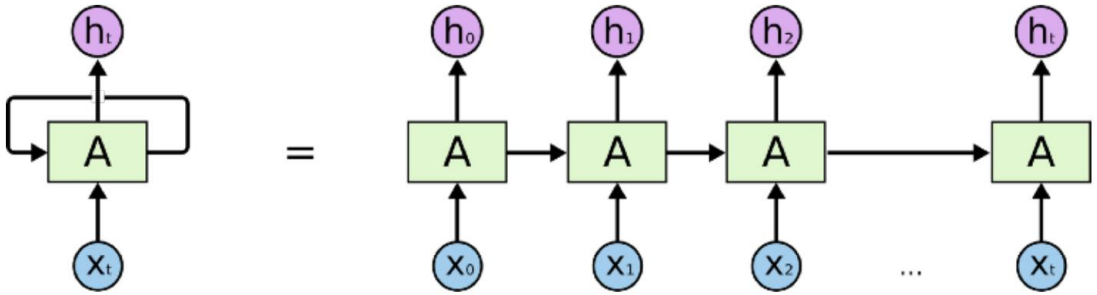


(Animation by Vincent Dumoulin)



(Source: Wikipedia)

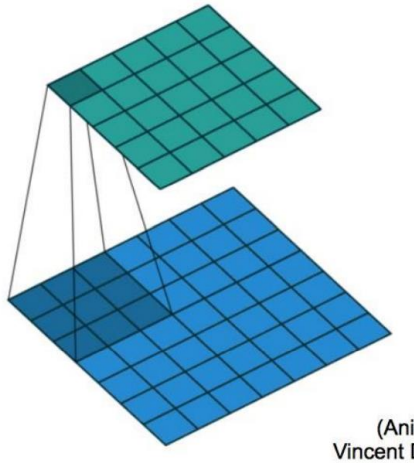
Recurrent neural networks (RNNs)



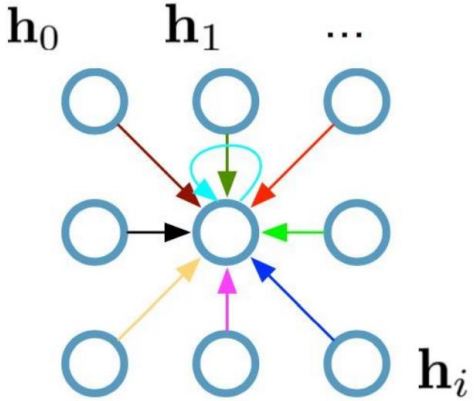
(Source: Christopher Olah's blog)

Recap: CNNs on Grids

Single CNN layer with 3x3 filter:



(Animation by Vincent Dumoulin)



Update for a single pixel:

- Transform messages individually $\mathbf{W}_i\mathbf{h}_i$
- Add everything up $\sum_i \mathbf{W}_i\mathbf{h}_i$

$\mathbf{h}_i \in \mathbb{R}^F$ are (hidden layer) activations of a pixel/node

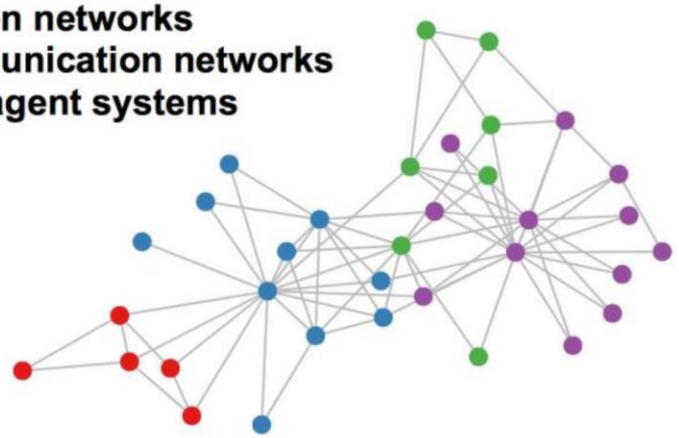
Full update:

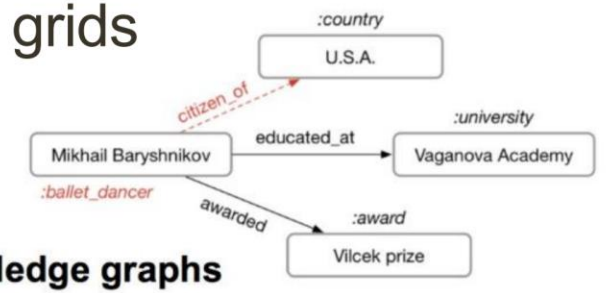
$$\mathbf{h}_4^{(l+1)} = \sigma\left(\mathbf{W}_0^{(l)}\mathbf{h}_0^{(l)} + \mathbf{W}_1^{(l)}\mathbf{h}_1^{(l)} + \cdots + \mathbf{W}_8^{(l)}\mathbf{h}_8^{(l)}\right)$$

Graph-structured data

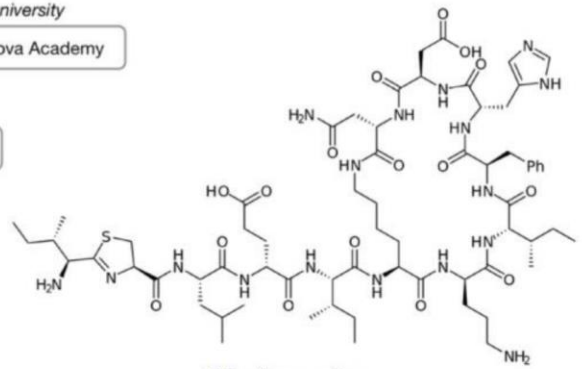
A lot of real-world data does not “live” on grids

Social networks
Citation networks
Communication networks
Multi-agent systems

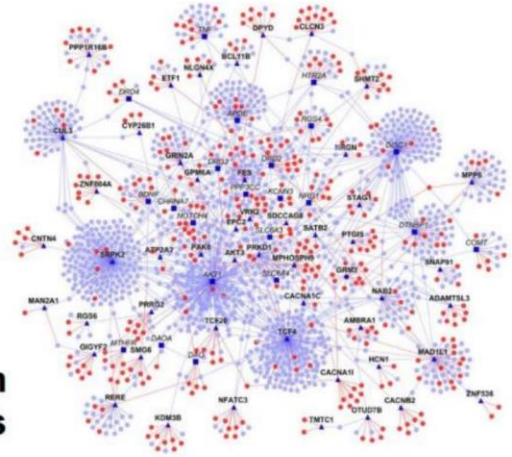




Knowledge graphs



Molecules



Protein interaction networks

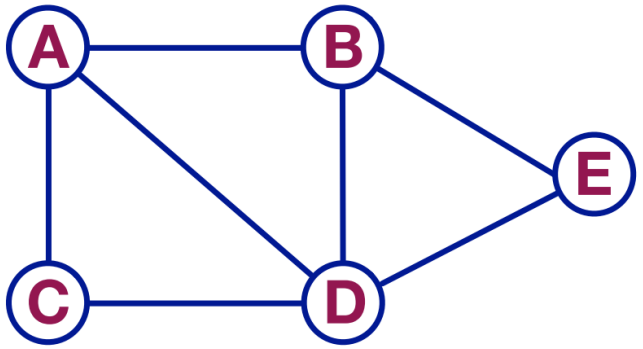
Road maps

Standard **CNN** and **RNN** architectures don’t work on this data

Graph-structured data

Graph: $G = (\mathcal{V}, \mathcal{E})$

Adjacency matrix: A



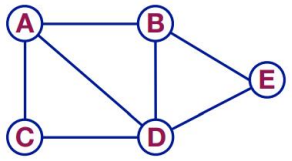
	A	B	C	D	E
A	0	1	1	1	0
B	1	0	0	1	1
C	1	0	0	1	0
D	1	1	1	0	1
E	0	1	0	1	0

Model wish list:

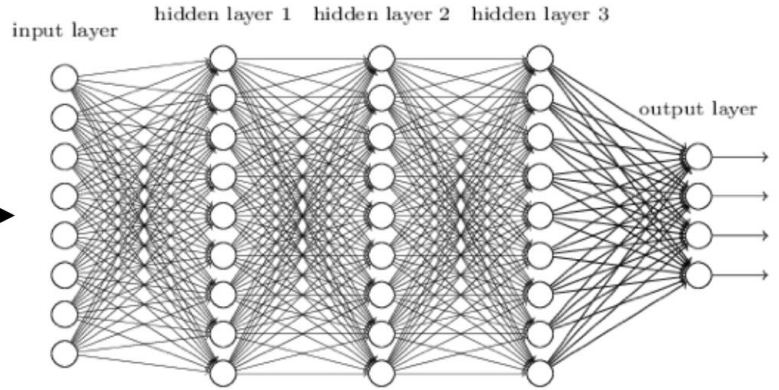
- Trainable in $\mathcal{O}(|\mathcal{E}|)$ time
- Applicable even if the input graph changes

A naïve approach

- Take adjacency matrix $\mathbf{A}$ and feature matrix $\mathbf{X}$
- Concatenate them $\mathbf{X}_{\text{in}} = [\mathbf{A}, \mathbf{X}]$
- Feed them into deep (fully connected) neural net
- Done?



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



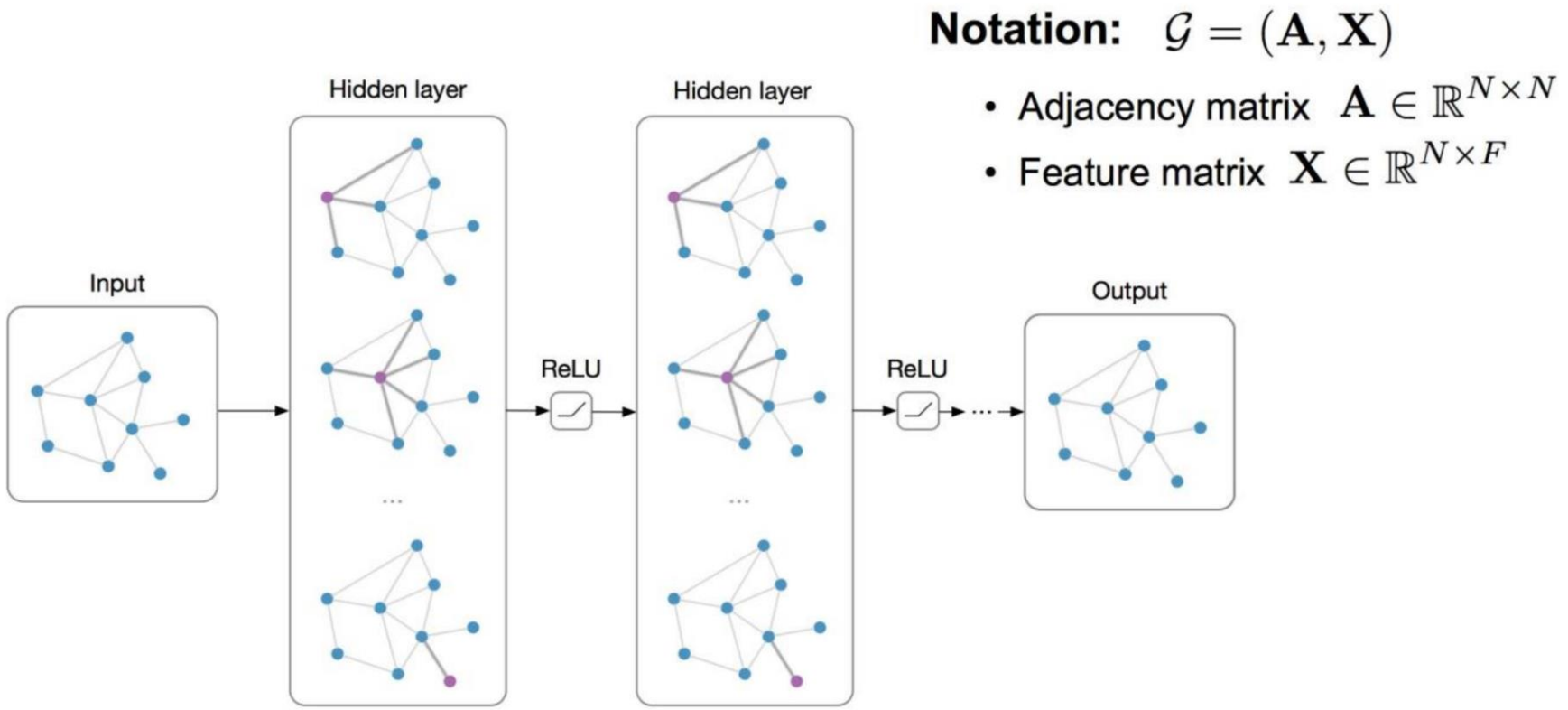
?

Problems:

- Huge number of parameters $\mathcal{O}(N)$
- Re-train if graph changes

Graph Neural Networks (GNNs)

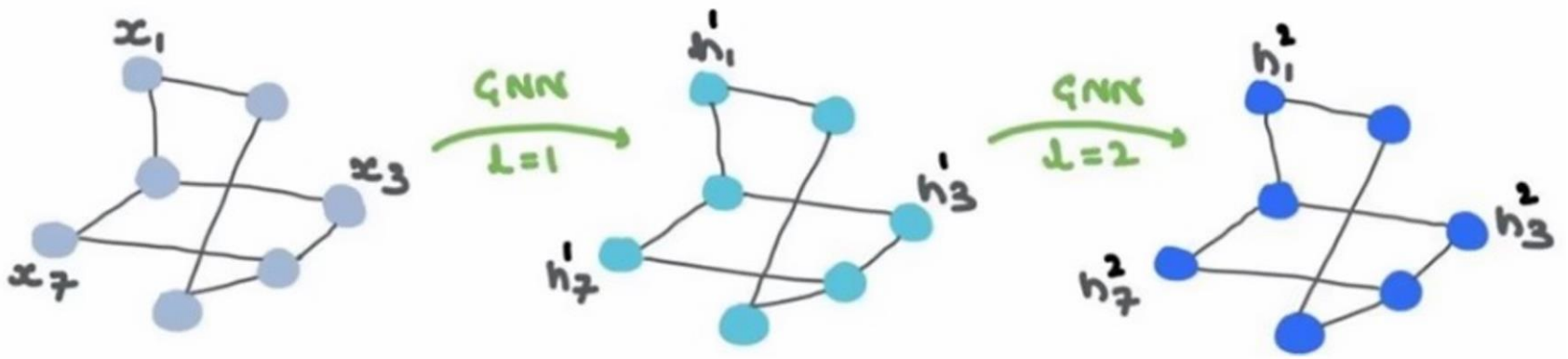
Graph neural networks (GNNs)



- **Main idea**: Pass messages between pairs of nodes and agglomerate
- **Alternative interpretation**: Pass messages between nodes to refine the representation of nodes and edges

Graph neural networks (GNNs)

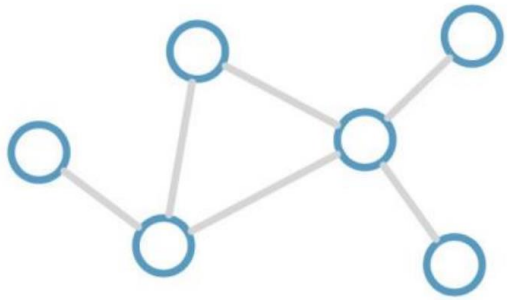
- GNNs can also have multiple layers
  - The essence of GNN is to update various feature components
  - Both the input and the output are features, and the adjacency matrix remains unchanged
- What can we do with the output features?
  - Can use the combined features of various points for graph classification
  - Can classify individual nodes or edges
  - In reality, it is just about extracting features from the graph structure; what we ultimately do with them depends on our own objectives



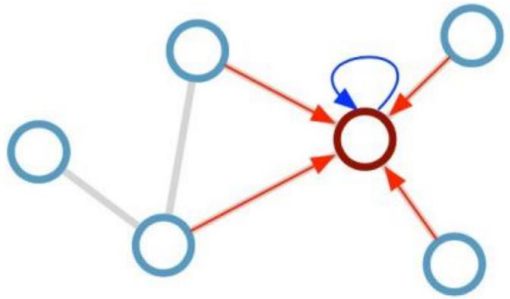
Graph Convolutional Networks (GCNs)

Kipf & Welling (ICLR 2017), related previous works by Duvenaud et al. (NIPS 2015) and Li et al. (ICLR 2016)

Consider this undirected graph:



Calculate update for node in red:



Desirable properties:

- Weight sharing over all locations
- Invariance to permutations
- Linear complexity O(E)
- Applicable both in transductive and inductive settings

Update rule:

$$\mathbf{h}_i^{(l+1)} = \sigma\left(\mathbf{h}_i^{(l)}\mathbf{W}_0^{(l)} + \sum_{j\in\mathcal{N}_i}\frac{1}{c_{ij}}\mathbf{h}_j^{(l)}\mathbf{W}_1^{(l)}\right)$$

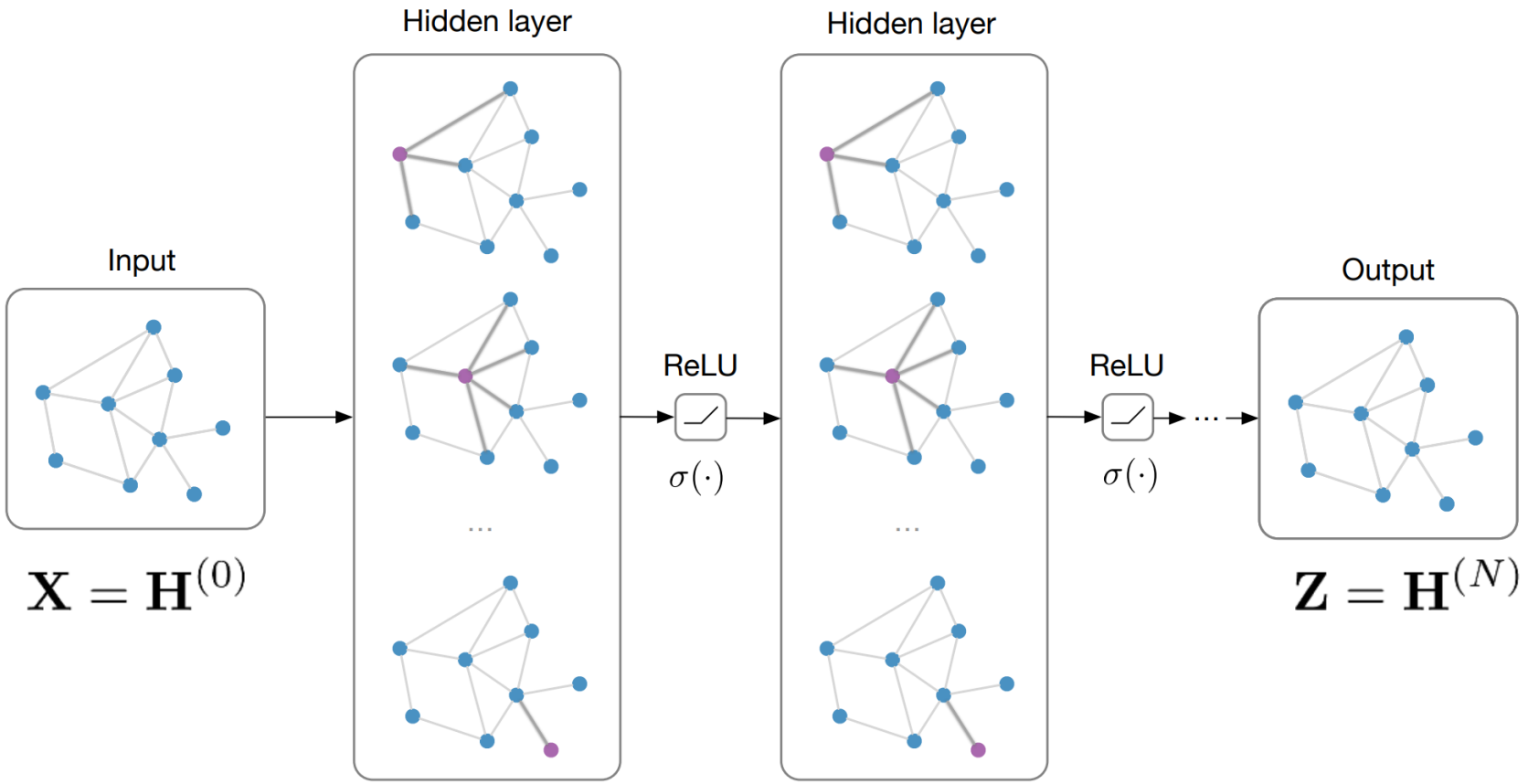
GCNs with first-order message passing

Scalability: subsample messages [Hamilton et al., NIPS 2017]

$\mathcal{N}_i$: neighbor indices c_{ij} : norm. constant (fixed/trainable)

GCN model architecture

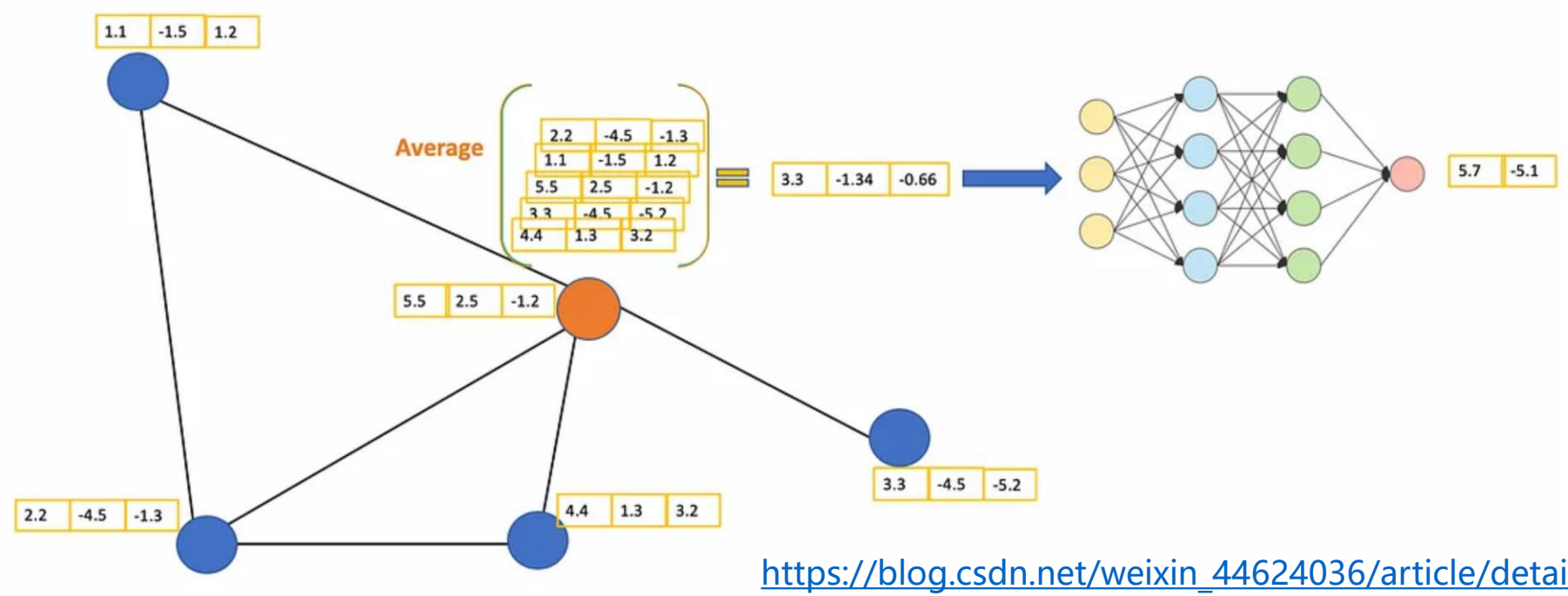
Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



$$\mathbf{H}^{(l+1)} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

[Kipf & Welling, ICLR 2017]

Fundamental idea of GCN

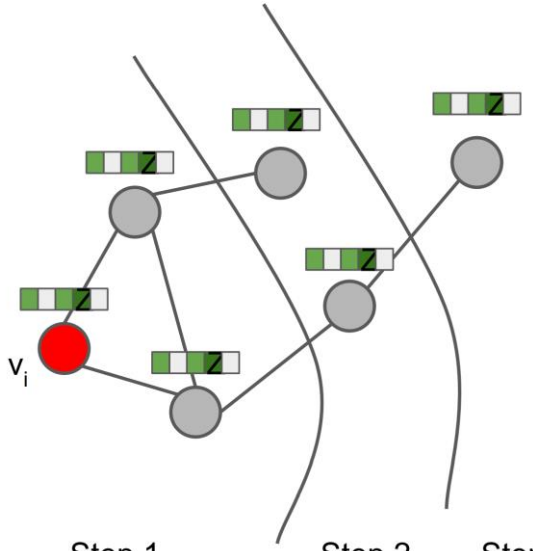


https://blog.csdn.net/weixin_44624036/article/details/131622229

- For the orange node, calculate its features by averaging the features of its neighbors (including itself) and then passing them through a neural network
- Similar to convolutional neural networks (CNNs), GCNs can have multiple layers.
  - In each layer, the input remains the node features
  - These features are then passed to the next layer along with the network structure graph for further computation

Graph view of GCN model

- At every iteration, the model aggregates information from one hop deeper
- The number of layers in GCN
  - In theory, more layers are better
  - But in real-world graphs, you might not need that many layers
  - For example, in social networks you can reach anywhere in the entire world with just 6 hops
  - Therefore, most GCNs do not have many layers
  - Across multiple graph datasets, it has been found that 2 to 3 layers are relatively suitable, and having too many layers may even worsen the performance



Kipf & Welling, ICLR 2017

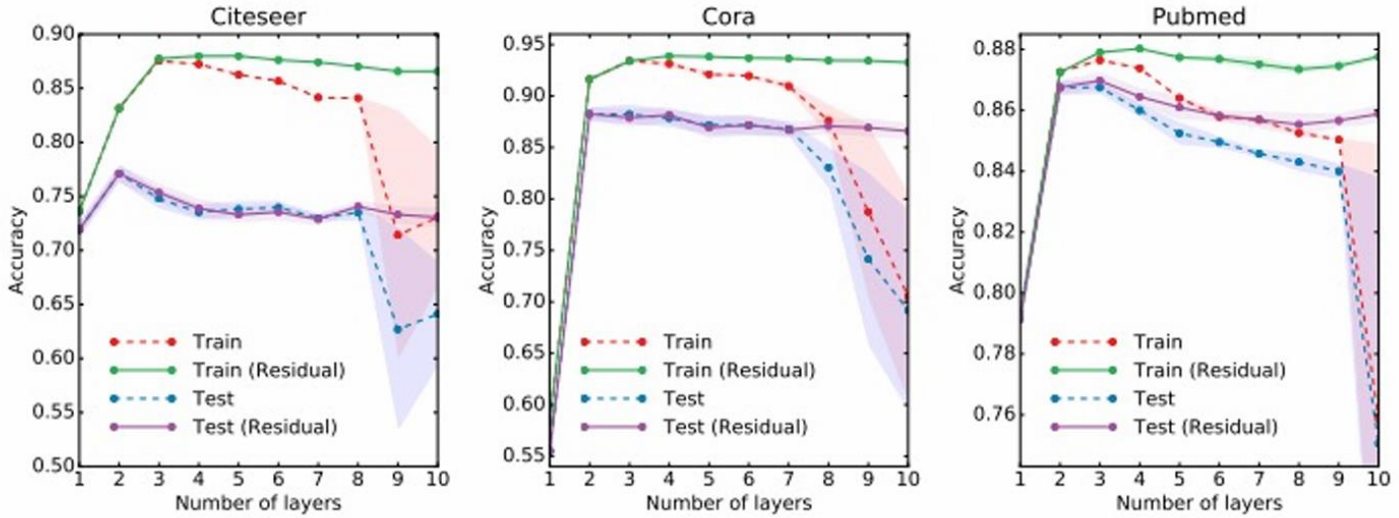


Figure 5: Influence of model depth (number of layers) on classification performance. Markers denote mean classification accuracy (training vs. testing) for 5-fold cross-validation. Shaded areas denote standard error. We show results both for a standard GCN model (dashed lines) and a model with added residual connections (He et al., 2016) between hidden layers (solid lines).

Limitations of GCNs

- Fixed aggregation function
- Lack of scalability
- Treats all graph edges equally
- Effective depth is limited

GCN extension: GraphSAGE

SAGE = **SA**mple Aggre**G**at**E**

Algorithm 1: GraphSAGE embedding generation (i.e., forward propagation) algorithm

Input : Graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth K; weight matrices $\mathbf{W}^k, \forall k \in \{1, ..., K\}$; non-linearity σ; differentiable aggregator functions $\text{AGGREGATE}_k, \forall k \in \{1, ..., K\}$; neighborhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$

$\mathbf{h}_v^0 \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$;

for $k = 1...K$ **do**

  for $v \in \mathcal{V}$ **do**

    $\mathbf{h}_{\mathcal{N}(v)}^k \leftarrow \text{AGGREGATE}_k(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\})$;

    $\mathbf{h}_v^k \leftarrow \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^k)\right)$

  end

  $\mathbf{h}_v^k \leftarrow \mathbf{h}_v^k / \|\mathbf{h}_v^k\|_2, \forall v \in \mathcal{V}$

end

$\mathbf{z}_v \leftarrow \mathbf{h}_v^K, \forall v \in \mathcal{V}$

Flexible aggregation (e.g. average or LSTM)

Stochastic sampling

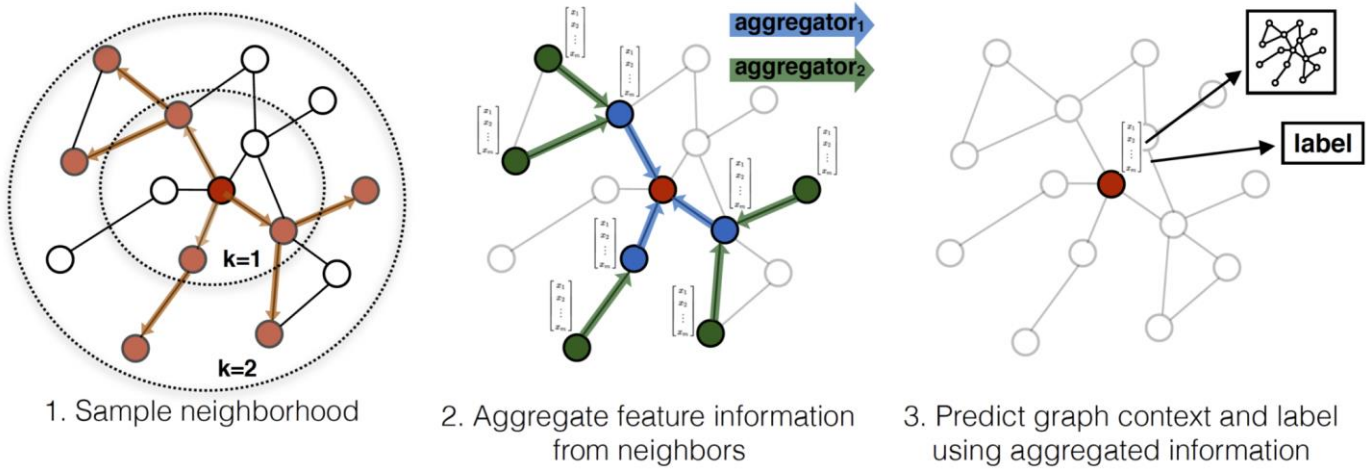
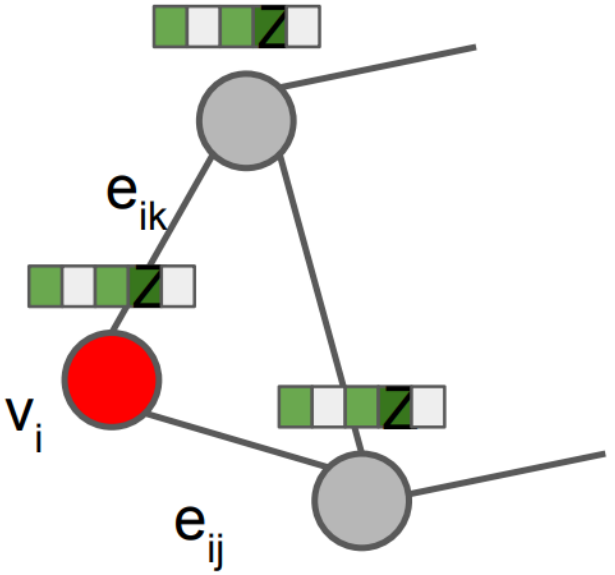


Figure 1: Visual illustration of the GraphSAGE sample and aggregate approach.

Hamilton, et al., NeurIPS 2017

GNNs with Attention (GATs)

- **Problem**: GCNs assume that the importance of the edges in the graph are equal.
- **Idea**: What if we add a learnable attention weight α_{ij} for each edge?

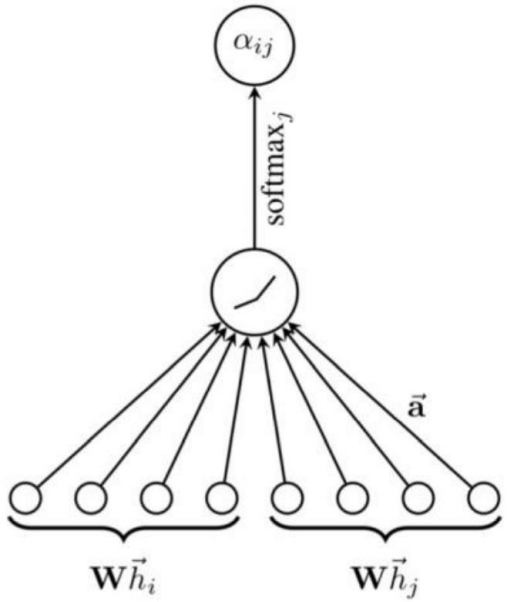


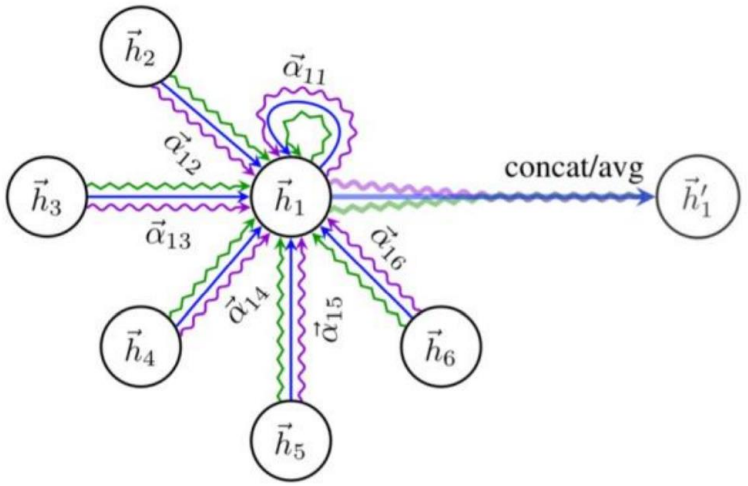
$$\alpha_{ij} = \text{softmax}_j(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})}.$$

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T [\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k \in \mathcal{N}_i} \exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T [\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_k]\right)\right)}$$

GNNs with Attention (GATs)

Monti et al. (CVPR 2017), Hoshen (NIPS 2017), Veličković et al. (ICLR 2018)





[Figure from Veličković et al. (ICLR 2018)]

Pros:

- No need to store intermediate edge-based activation vectors (when using dot-product attn.)
- Slower than GCNs but faster than GNNs with edge embeddings

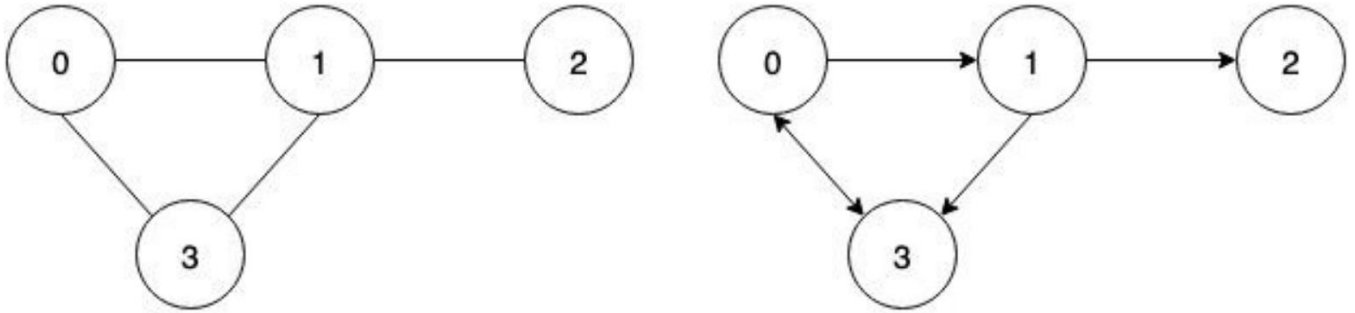
Cons:

- (Most likely) less expressive than GNNs with edge embeddings
- Can be more difficult to optimize

$$\vec{h}'_i = \sigma\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{j\in\mathcal{N}_i}\alpha_{ij}^k\mathbf{W}^k\vec{h}_j\right) \qquad \alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i\|\mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i\|\mathbf{W}\vec{h}_k]\right)\right)}$$

Variational Graph Auto-Encoders (VGAEs)

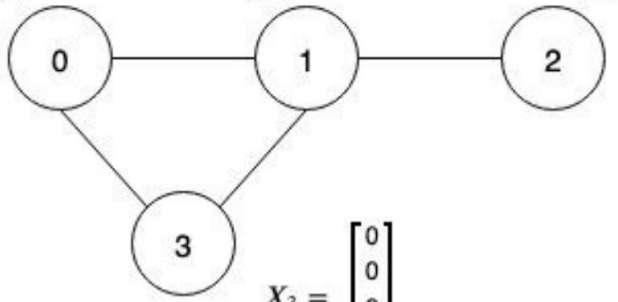
- **Main idea**: To apply the idea of Variational Autoencoder (VAE) to graph-structured data, to generate new graphs or reason about graphs (Kipf & Welling, NIPS Bayesian Deep Learning Workshop, 2016)



$$A_1 = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix} \qquad A_2 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}$$

- Input data:
  - Adjacency matrix **A**: topology of the graph
  - Feature matrix **X**: features of each node from the input graph

$$X_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad X_1 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad X_2 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

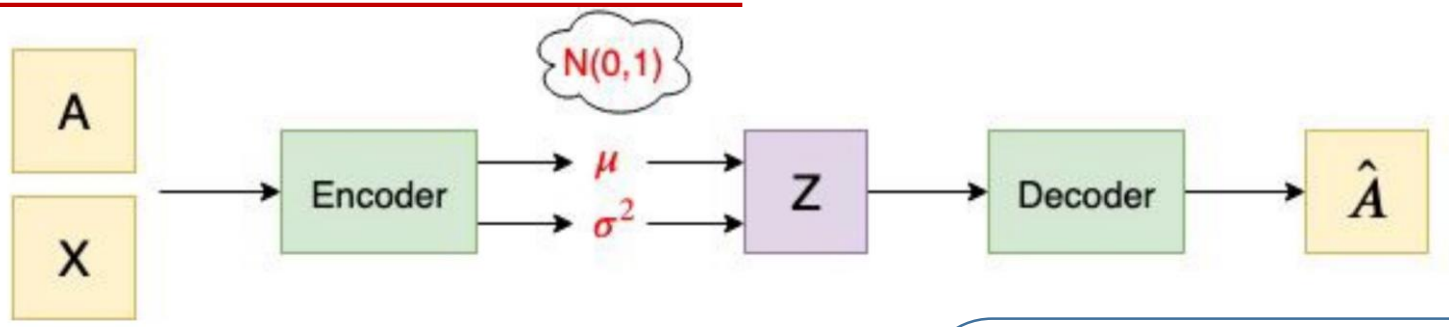


$$X_3 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

https://towardsdatascience.com/tutorial-on-variational-graph-auto-encoders-da9333281129

Architecture of VGAEs



- **Encoder (inference model)**: A 2-layer graph convolutional network (GCN)
- GCN layer 1 outputs a lower-dimensional feature matrix: $\bar{X} = GCN(X, A) = ReLU(\tilde{A}XW_0)$

$$\tilde{A} = D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$$

- GCN layer 2 outputs the two parameters of a Normal distribution:

$$GCN(X, A) = \tilde{A}ReLU(\tilde{A}XW_0)W_1$$

$$\mu = GCN_\mu(X, A)$$

$$log\sigma^2 = GCN_\sigma(X, A)$$

- **Decoder (generative model)**: An inner product between latent variable Z and its transpose, which is a reconstructed adjacency matrix $\hat{A}$

$$\hat{A} = \sigma(zz^T)$$

Calculate Z using parameterization trick

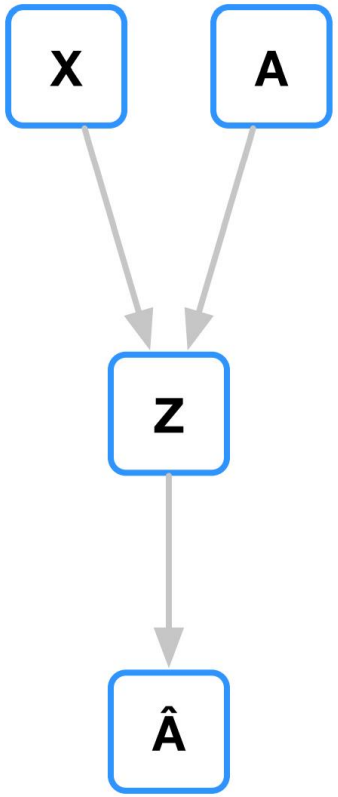
$$Z = \mu + \sigma * \epsilon$$

where $\epsilon \sim N(0, 1)$

https://towardsdatascience.com/tutorial-on-variational-graph-auto-encoders-da9333281129

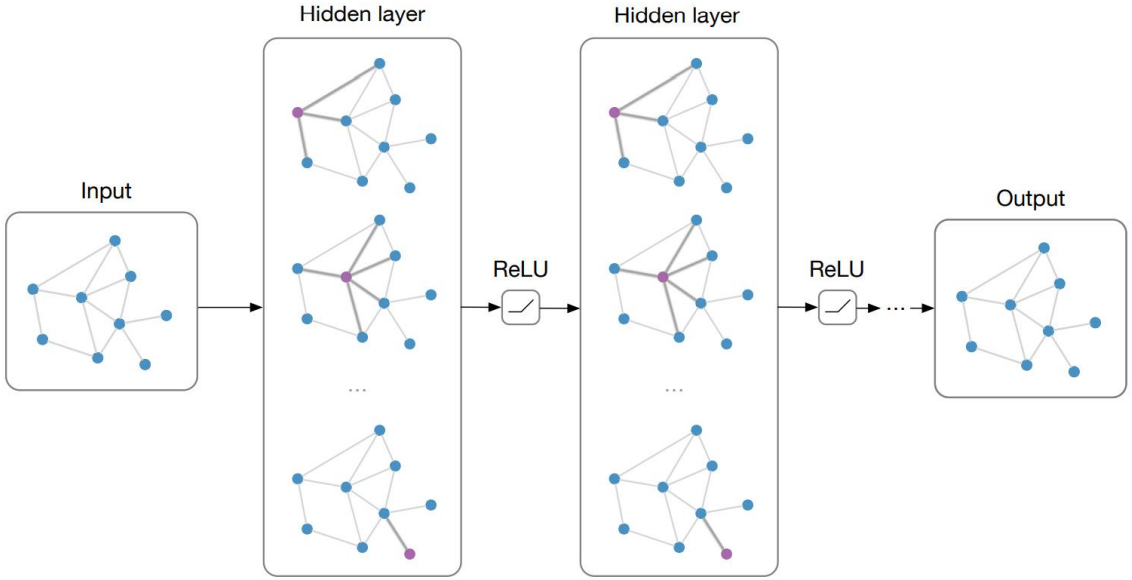
Link prediction with GAEs

Kipf & Welling, NIPS Bayesian Deep Learning Workshop, 2016



GAE

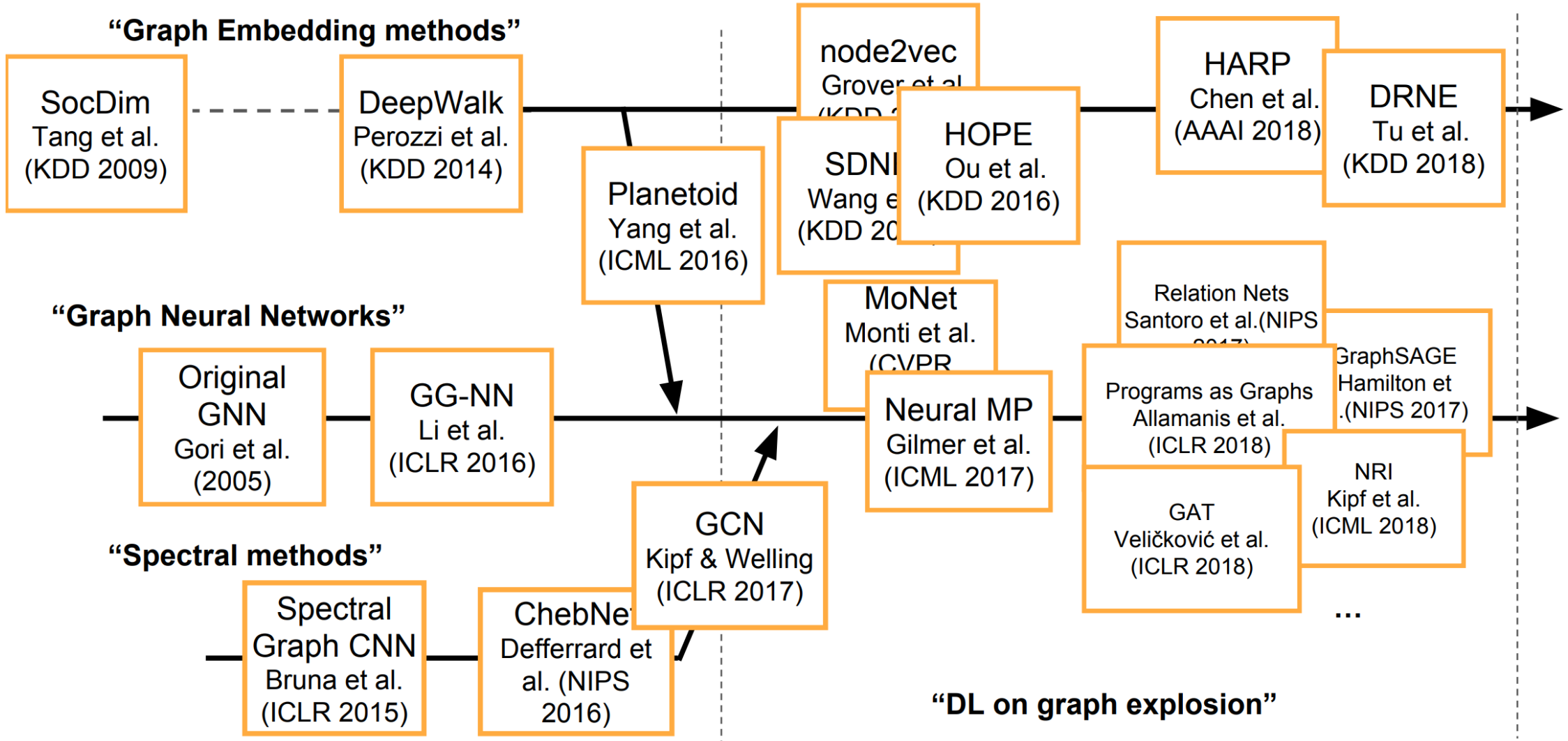
Encoder $\mathbf{Z} = \mathrm{GCN}(\mathbf{X}, \mathbf{A})$



Decoder $\hat{\mathbf{A}} = \sigma(\mathbf{Z}\mathbf{Z}^\top)$

This is a **non-probabilistic graph auto-encoder (GAE)** model, a variant of VGAE

A brief history of GNNs



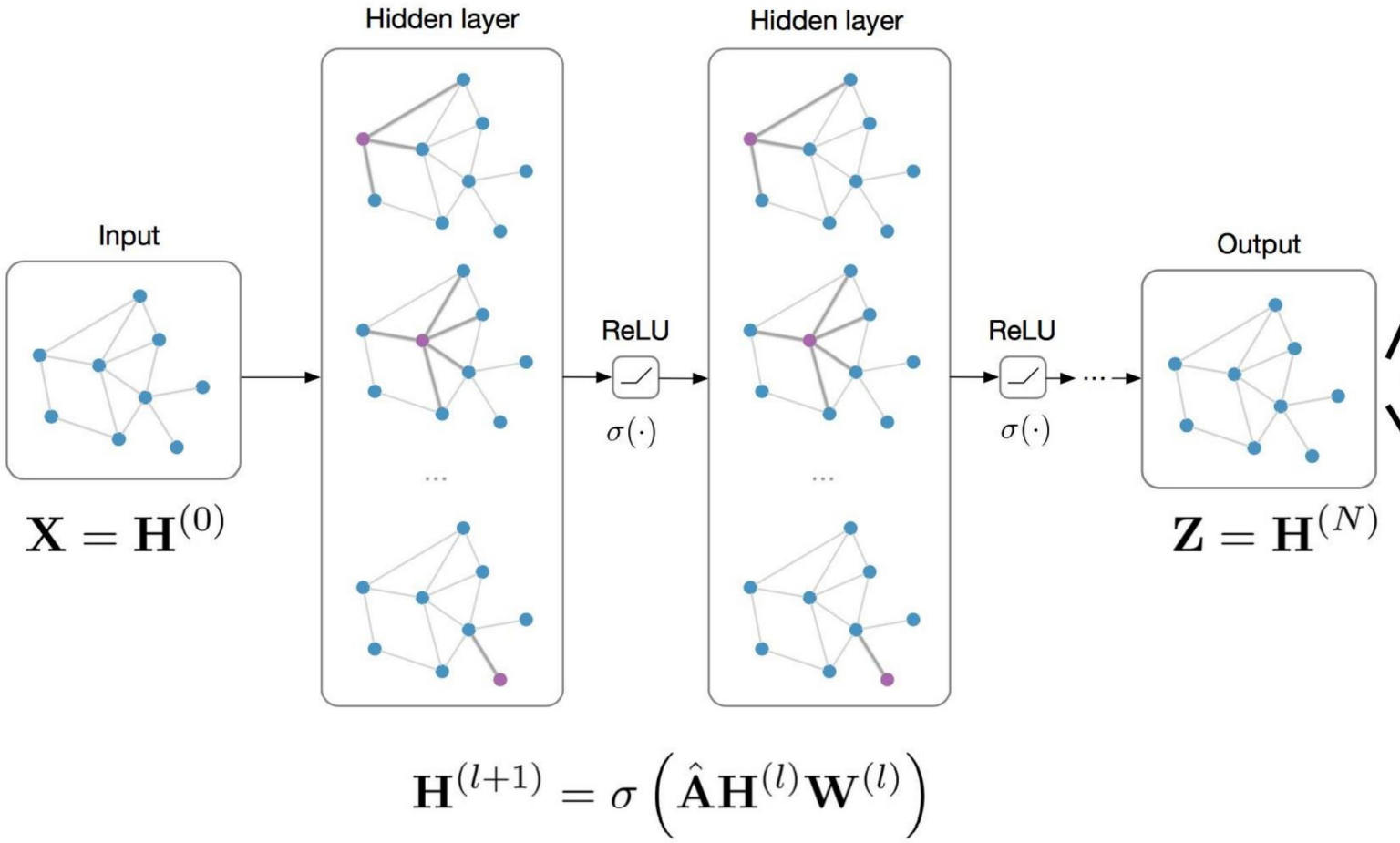
(slide inspired by Thomas Kipf's talk on GNNs)

Slide by Bryan Perozzi (KDD 2018 Tutorial)

Applications of GNNs

Classification and link prediction with GNNs

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



Node classification:

$\mathrm{softmax}(\mathbf{z_n})$

e.g. Kipf & Welling (ICLR 2017)

Graph classification:

$\mathrm{softmax}(\sum_n \mathbf{z_n})$

e.g. Duvenaud et al. (NIPS 2015)

Link prediction:

$p(A_{ij}) = \sigma(\mathbf{z_i^T z_j})$

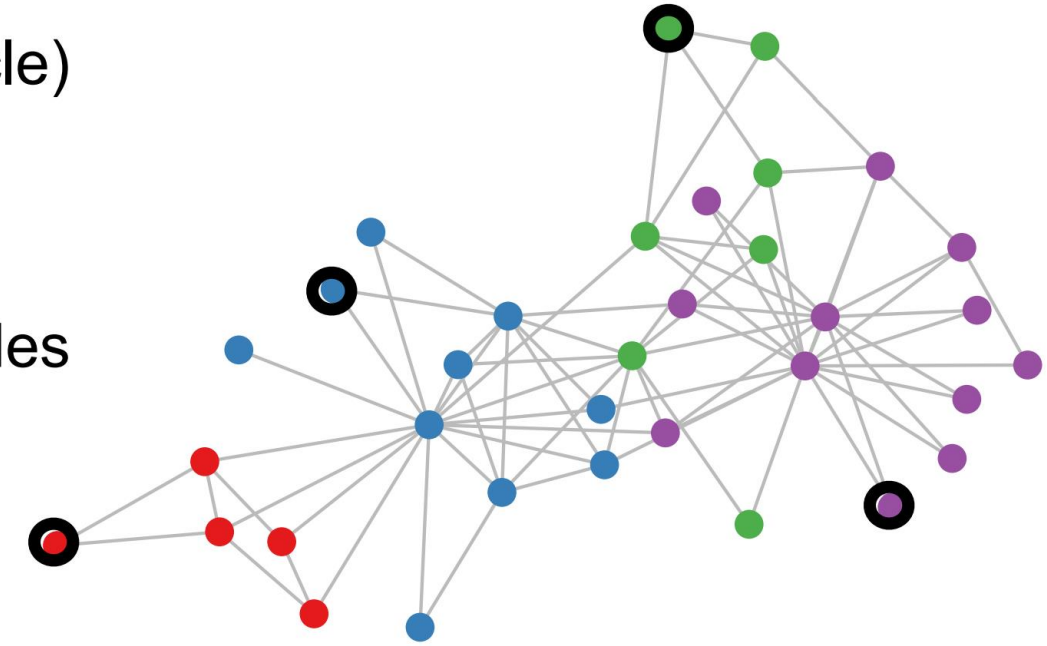
Kipf & Welling (NIPS BDL 2016)

"Graph Auto-Encoders"

Semi-supervised classification on graphs

Setting:

Some nodes are labeled (black circle)
All other nodes are unlabeled



Task:

Predict node label of unlabeled nodes

Standard approach:

graph-based regularization [Zhu et al., 2003]

$$\mathcal{L} = \mathcal{L}_0 + \lambda \mathcal{L}_{\text{reg}} \quad \text{with} \quad \mathcal{L}_{\text{reg}} = \sum_{i,j} A_{ij} \| f(X_i) - f(X_j) \|^2$$

assumes: connected nodes likely to share same label

Semi-supervised classification on graphs

- **Embedding-based approaches**
- Two-step pipeline:
  1) Get an embedding for every node
  2) Train a classifier on the node embedding
- **Examples**: DeepWalk [Perozzi et al., 2014], node2vec [Grover & Leskovec, 2016]
- **Issue**: Embeddings are not optimized for classification
- **Idea**: Train a graph-based classifier end-to-end using GCN

  Evaluate loss on labeled nodes only:

$$\mathcal{L} = -\sum_{l \in \mathcal{Y}_L} \sum_{f=1}^{F} Y_{lf} \ln Z_{lf}$$

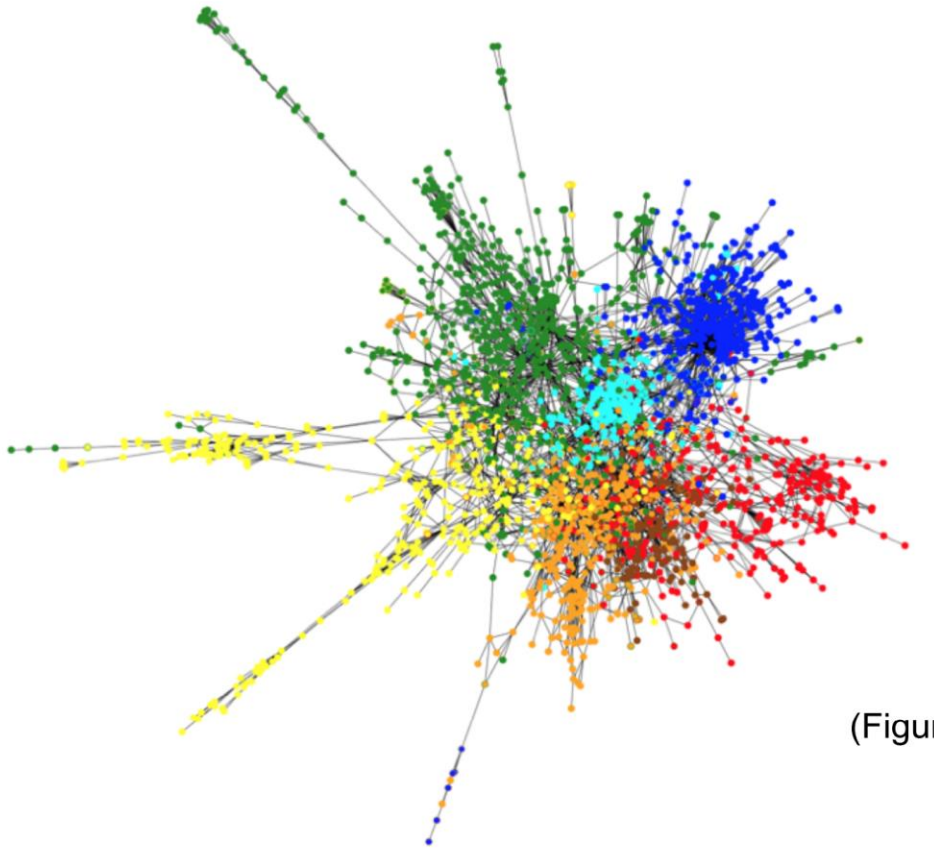
$\mathcal{Y}_L$ set of labeled node indices

$\mathbf{Y}$ label matrix

$\mathbf{Z}$ GCN output (after softmax)

Application: Classification on citation networks

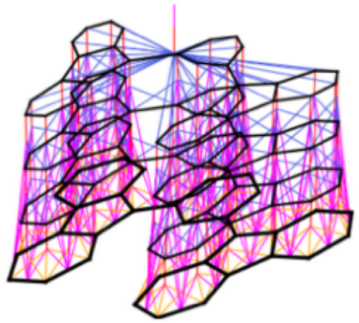
- **Input**: Citation networks (nodes are papers, and edges are citation links, optionally with bag-of-words features on the nodes)
- **Target**: Paper category (e.g. stat.ML, cs.LG, …)

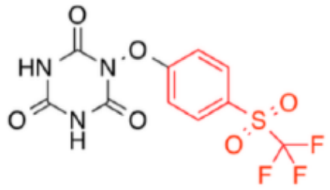


(Figure from: Bronstein, Bruna, LeCun, Szlam, Vandergheynst, 2016)

Other recent applications

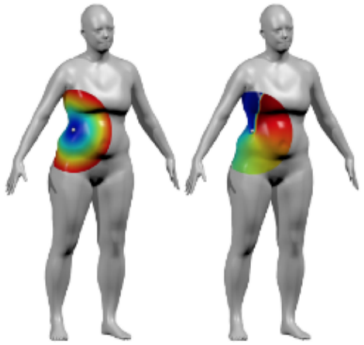
Molecules



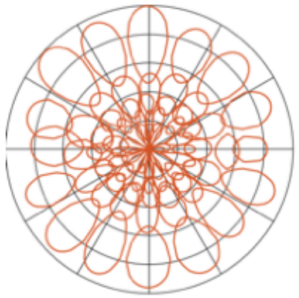


[Duvenaud et al., NIPS 2015]

Shapes



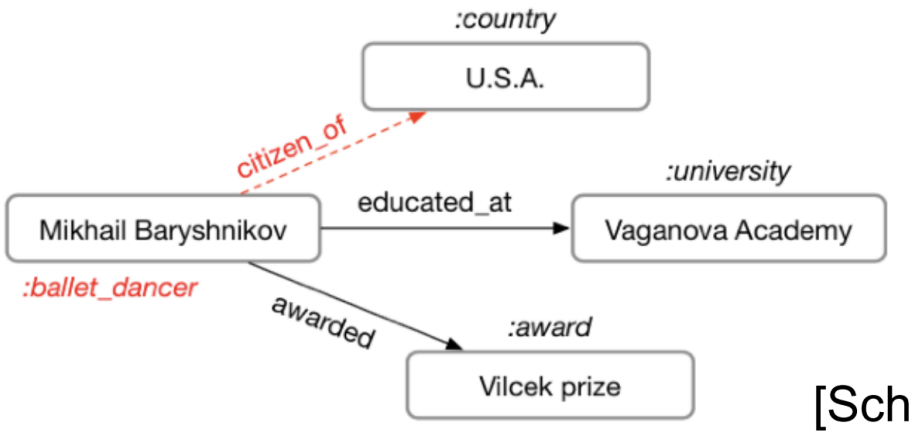
Polar coordinates ρ, θ



MoNet

[Monti et al., 2016]

Knowledge Graphs

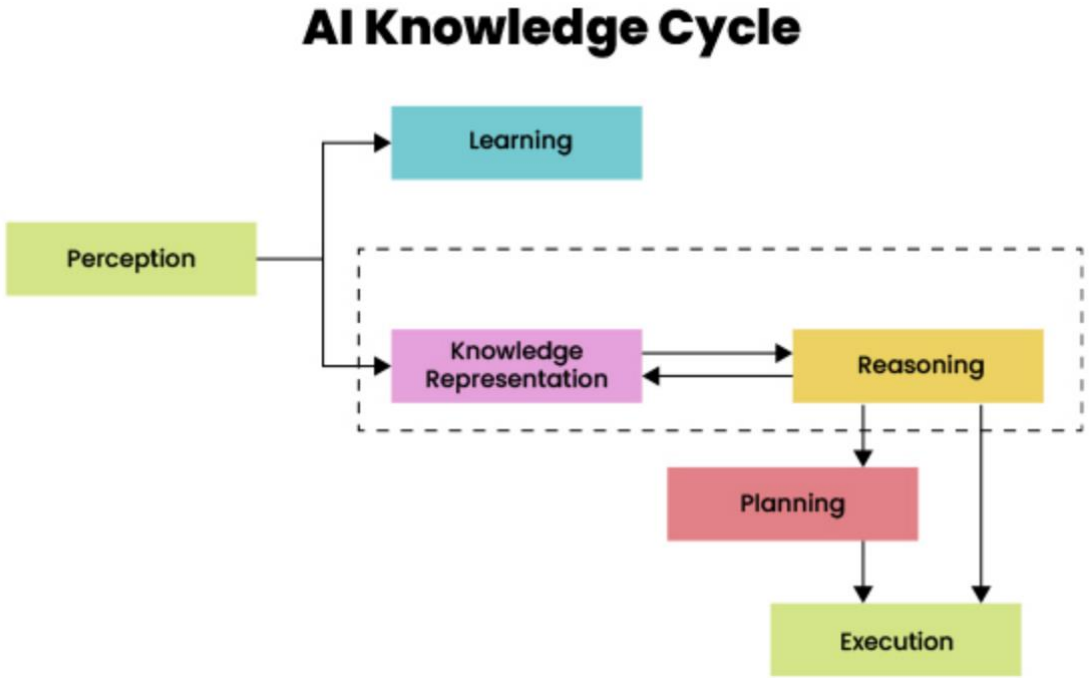


[Schlichtkrull et al., 2017]

Knowledge Graph (KG)

Background

- Incorporating human knowledge is an important direction of AI
- Knowledge representation and reasoning, inspired by human problem solving, can enable AI systems to solve complex tasks (e.g. medical diagnosis)
- Recently, knowledge graphs as a form of structured human knowledge have drawn great attention from both academia and industry



https://thinkpalm.com/blogs/knowledge-representation-in-ai-and-its-significance-in-business/

History of knowledge representation

- Knowledge representation has a long history in logic and AI

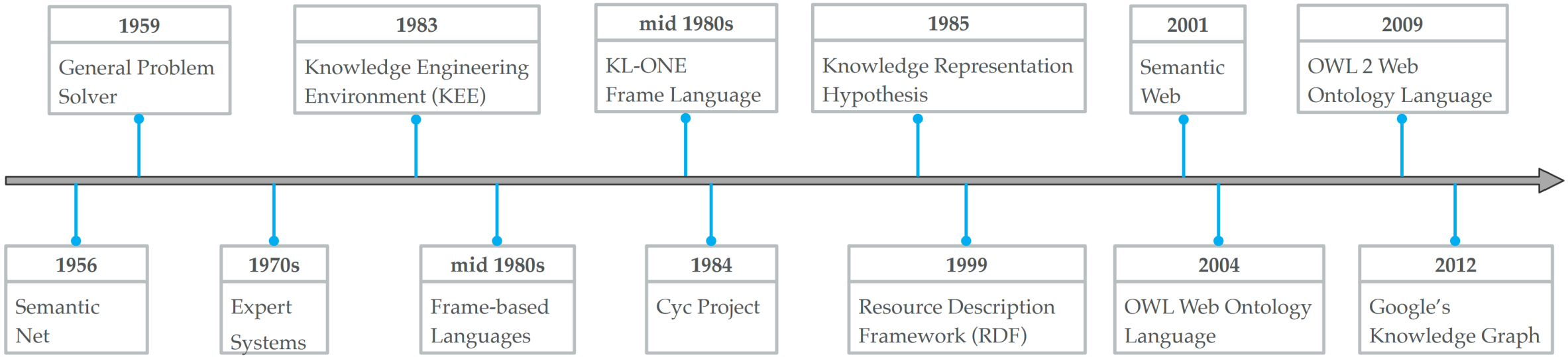


Figure source: Ji et al. A survey on knowledge graphs: representation, acquisition and applications, IEEE Trans. Neural Networks and Learning Systems, 2021

Definition of knowledge graph (KG)

- There is no wide-accepted formal definition as yet.

- **Definition 1** (Ehrlinger and Wöß, 2016): A **knowledge graph** acquires and integrates information into an ontology and applies a reasoner to derive new knowledge.
  - **Ontology**: A set of concepts and categories in a subject area or domain that shows their properties and the relations between them.

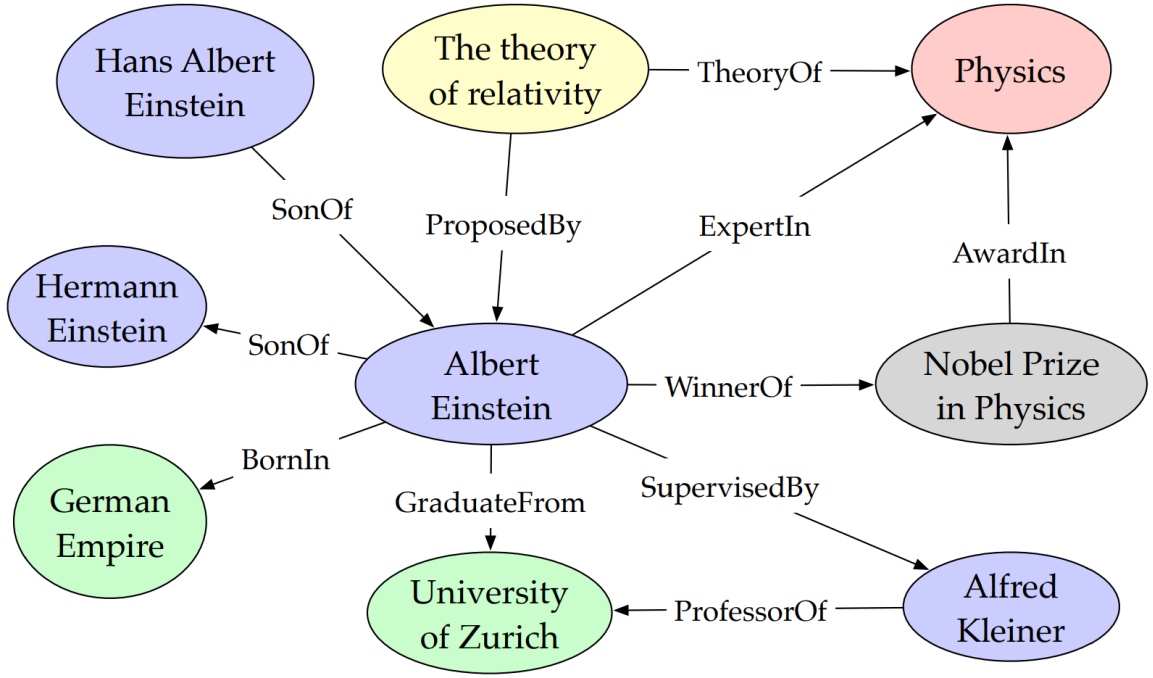
- **Definition 2** (Wang et al. 2017): A **knowledge graph** is a multi-relational graph composed of entities and relations which are regarded as nodes and different types of edges, respectively.

- **Definition 3** (Ji et al. 2021): A **knowledge graph** is defined as $G = \{E, R, F\}$, where E is the set of entities, R is the set of relations, and F is the set of facts, respectively.
  - A **fact** is denoted as a triple $(h, r, t) \in F$.

Example of knowledge base and KG

(Albert Einstein, **BornIn**, German Empire)
(Albert Einstein, **SonOf**, Hermann Einstein)
(Albert Einstein, **GraduateFrom**, University of Zurich)
(Albert Einstein, **WinnerOf**, Nobel Prize in Physics)
(Albert Einstein, **ExpertIn**, Physics)
(Nobel Prize in Physics, **AwardIn**, Physics)
(The theory of relativity, **TheoryOf**, Physics)
(Albert Einstein, **SupervisedBy**, Alfred Kleiner)
(Alfred Kleiner, **ProfessorOf**, University of Zurich)
(The theory of relativity, **ProposedBy**, Albert Einstein)
(Hans Albert Einstein, **SonOf**, Albert Einstein)

(a) Factual triples in a knowledge base.

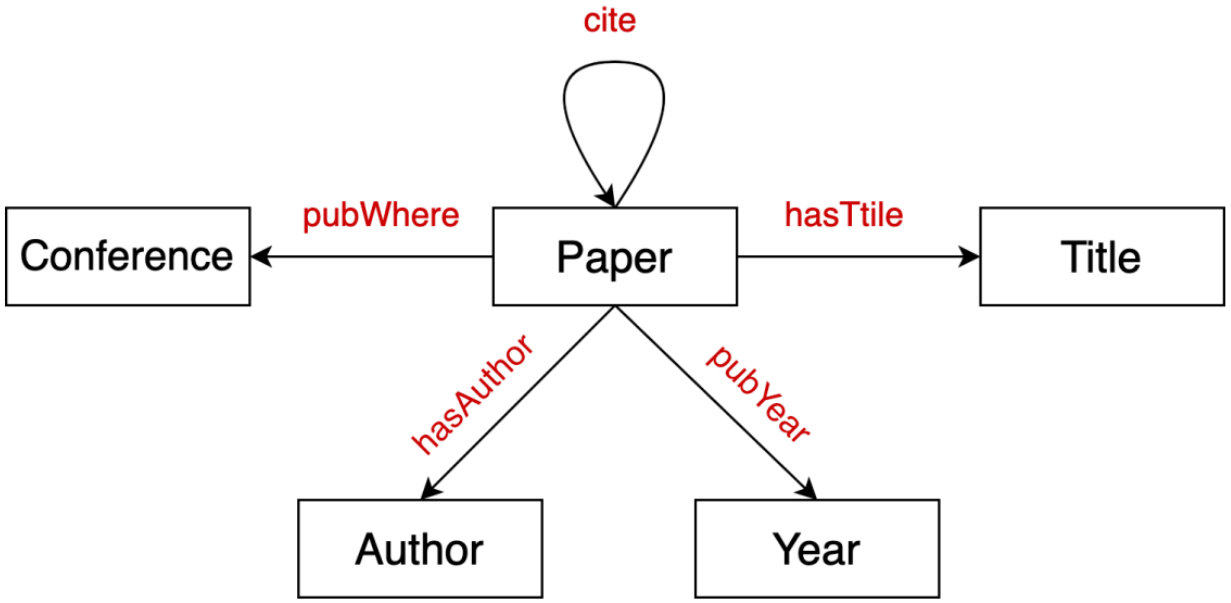


(b) Entities and relations in a knowledge graph.

Figure source: Ji et al. A survey on knowledge graphs: representation, acquisition and applications, IEEE Trans. Neural Networks and Learning Systems, 2021

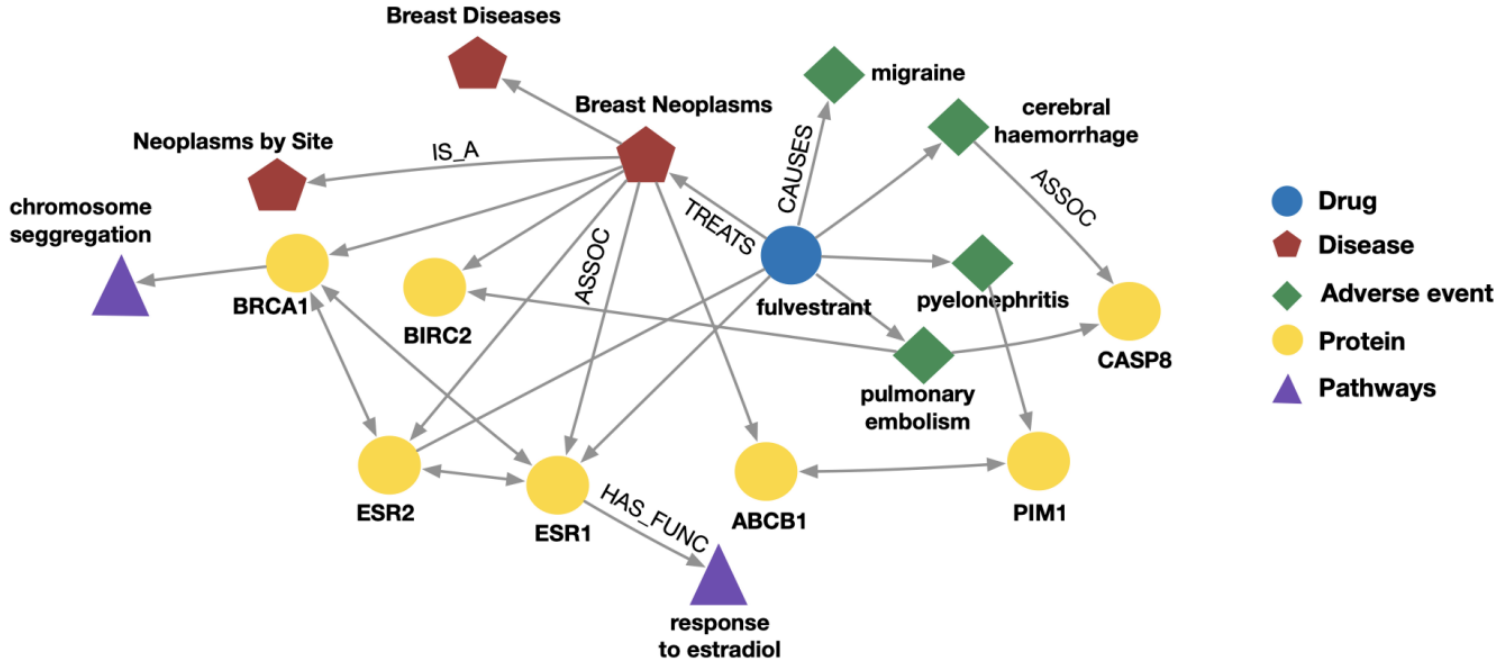
Example: Bibliographic networks

- **Node types**: paper, title, author, conference, year
- **Relation types**: pubWhere, pubYear, hasTitle, hasAuthor, cite



Example: Bio knowledge graphs

- **Node types**: drug, disease, adverse event, protein, pathways
- **Relation types**: has_func, causes, assoc, treats, is_a



A roadmap of research on KG

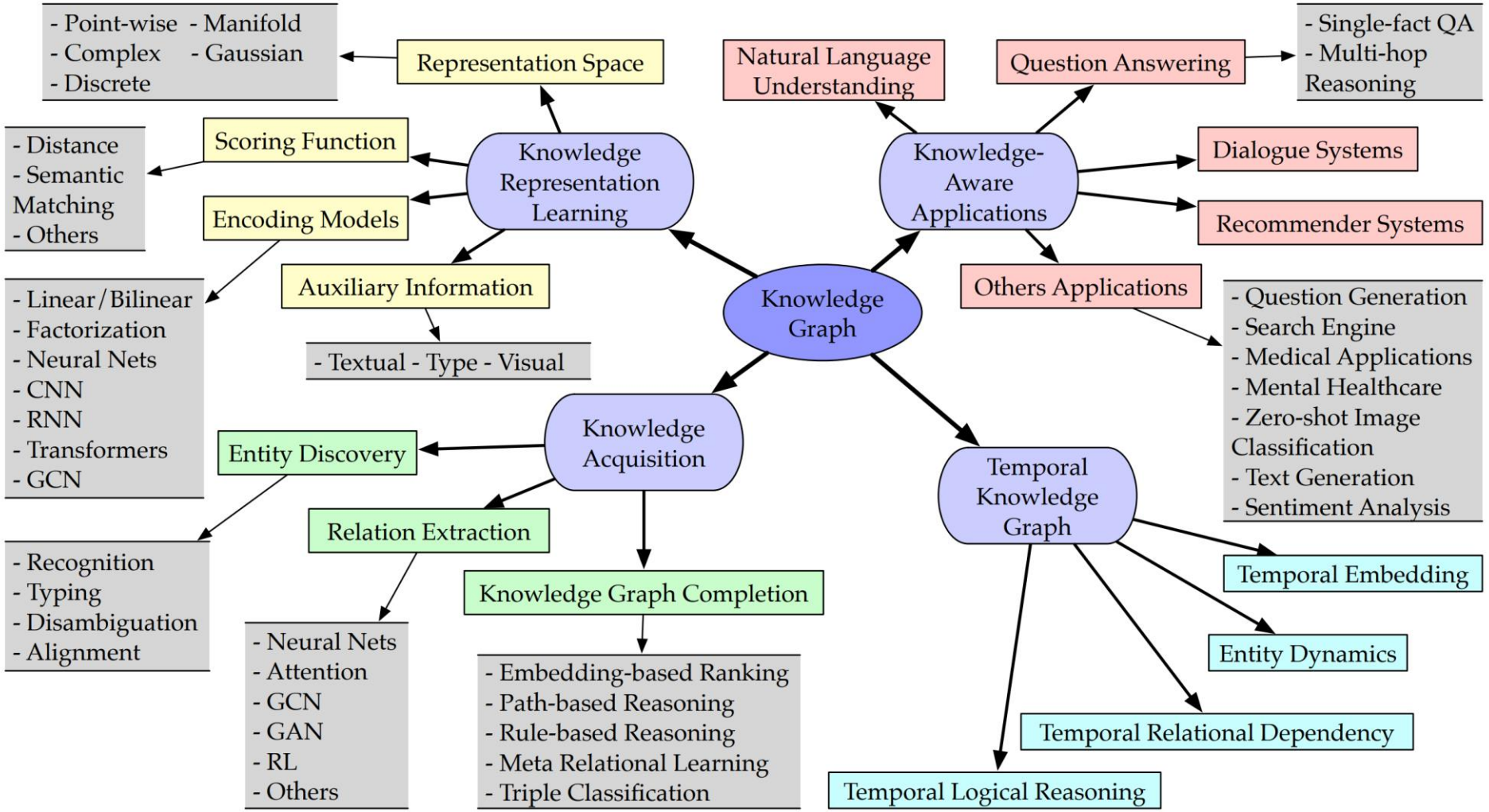
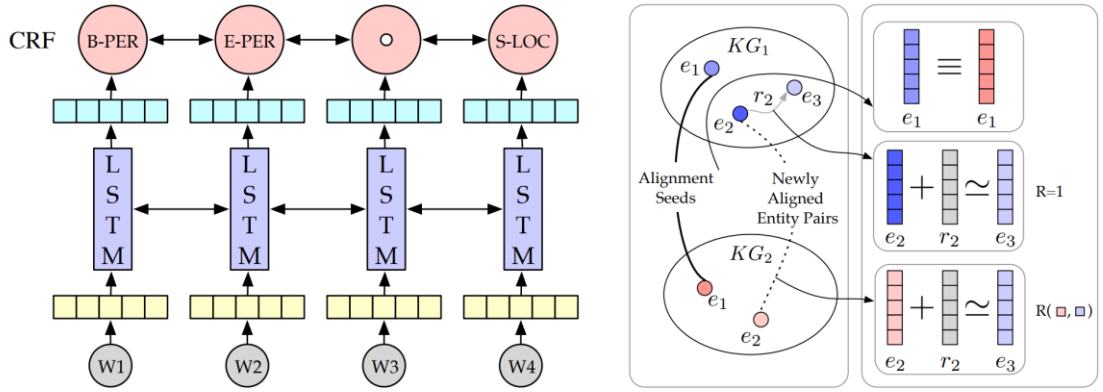


Figure source: Ji et al. A survey on knowledge graphs: representation, acquisition and applications, IEEE Trans. Neural Networks and Learning Systems, 2021

Building KG: Knowledge Acquisition

- **Knowledge graph completion** completes missing links between existing entities or infers entities given entity and relation queries
- **Entity discovery** acquires entity-oriented knowledge from text and fuses knowledge between knowledge graphs
- **Relation extraction** is to build large-scale KGs automatically by extracting unknown relational facts from plain text and adding them into KGs



(a) Entity recognition with LSTM-CRF [111]. (b) Entity alignment with IP-TransE [126].

Fig. 8: Illustrations of several entity discovery tasks.

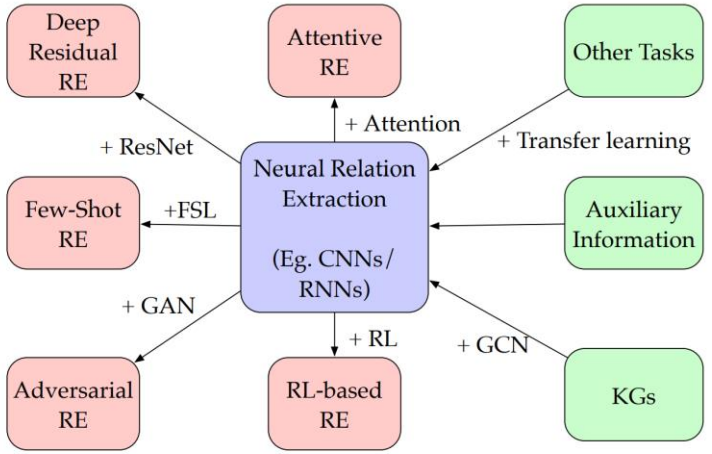
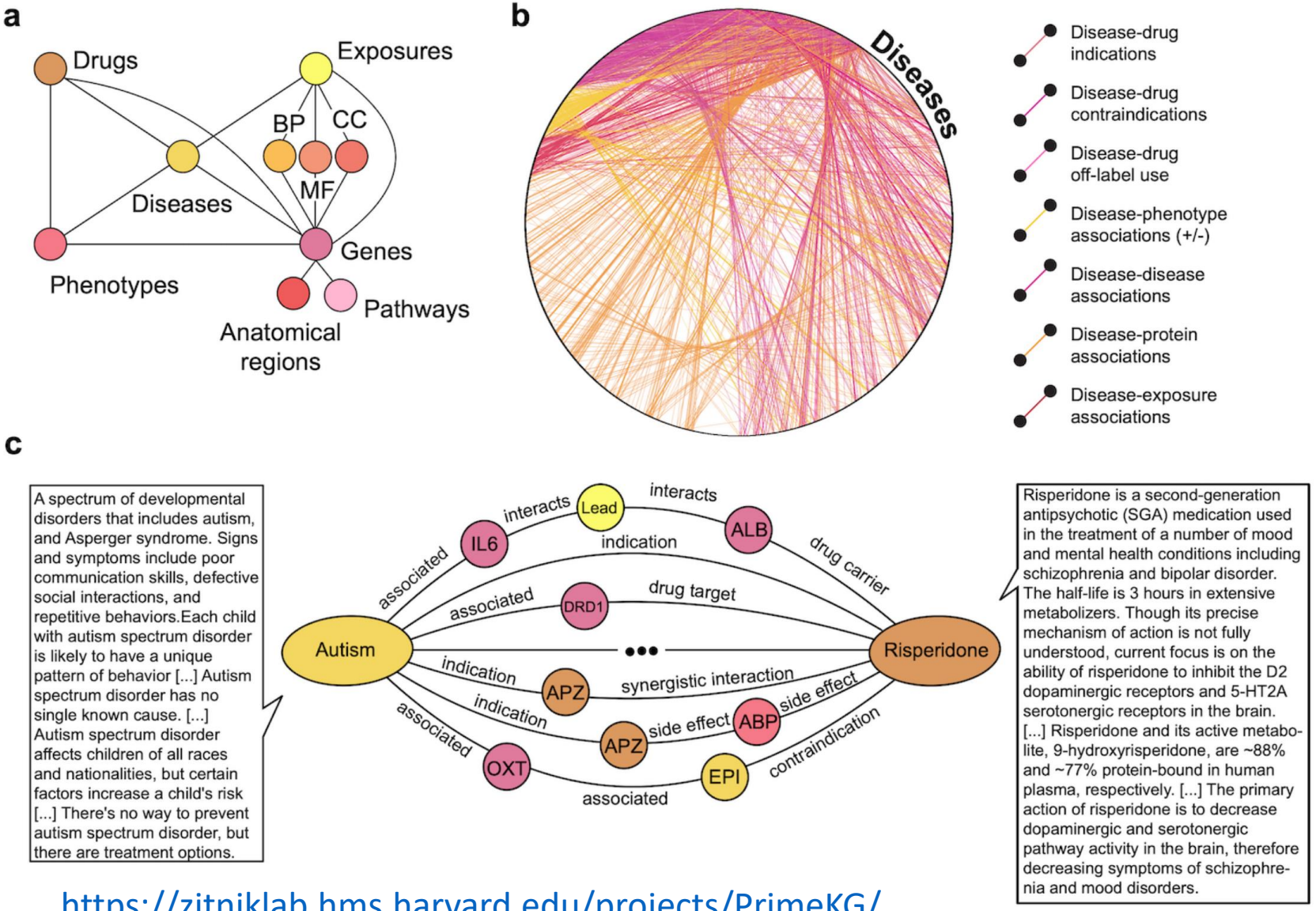


Fig. 9: An overview of neural relation extraction.

Figure source: Ji et al. IEEE Trans. Neural Networks and Learning Systems, 2021

PrimeKG: a KG for precision medicine

- **PrimeKG** is a precision medicine-oriented knowledge graph that provides a holistic view of diseases
  - 20 high-quality resources
  - 17,080 diseases
  - 4,050,249 relationships representing 10 major biological scales, including disease-associated protein perturbations, biological processes and pathways, etc.
- It supports drug-disease prediction



https://zitniklab.hms.harvard.edu/projects/PrimeKG/

Payal Chandak et al. Scientific Data, 2023

A list of biomedical KGs

表 5-5　现有生物医药知识图谱汇总

知识图谱	实体个数	实体种数	关系种数	数据源个数	发布年份
Hetionet[43]	47k	11	24	29	2017
DRKG[44]	97k	13	107	34	2020
BioKG[45]	105k	10	17	13	2020
PharmKG[46]	7.6k	3	29	7	2020
CKG[47]	16M	35	57	35	2020
PrimeKG [48]	129k	10	30	20	2023

- Table from text book Honglin Li et al. “Artificial Intelligence for Drug Design”, Chapter 5, p. 141

Summary

- In this lecture we have learned:
  - Graph convolutional networks (GCNs) apply ideas of CNNs to graphs, but have their own explanation (e.g. message passing and aggregation)
  - There are some powerful extensions of GCNs, e.g. GraphSAGE, edge embeddings, GNNs with attention
  - GNNs have mainly 3 categories of application tasks: node classification, graph classification, and link prediction
  - Graph Auto-encoders (GAEs) can do link prediction
  - Knowledge graphs as a technique of knowledge representation have been used in many domains including biomedicine

References

- Scarselli et al., “The Graph Neural Network Model”, IEEE Transactions on Neural Networks, 2009.
- Bruna et al., “Spectral Networks and Deep Locally Connected Networks on Graphs”, ICLR 2014.
- Kipf and Welling, “Variational Graph Auto-Encoders”, NIPS Bayesian Deep Learning Workshop, 2016.
- Kipf and Welling, “Semi-Supervised Classification with Graph Convolutional Networks”, ICLR 2017.
- Hamilton et al., “Inductive Representation Learning on Large Graphs”, NIPS 2017.
- Veličković et al., “Graph Attention Networks”, ICLR 2018.
- Kipf et al., “Neural Relational Inference for Interacting Systems”, ICML 2018.
- Wu et al., “A Comprehensive Survey on Graph Neural Networks”, IEEE Transactions on Neural Networks and Learning Systems, 2020.
- Sun et al., “Graph Convolutional Network for Computational Drug Development and Discovery”, Briefings in Bioinformatics, 2020.
- Shaoxiong Ji et al. “A Survey on Knowledge Graphs: Representation, Acquisition and Applications”, IEEE Transactions on Neural Networks and Learning Systems, 2021.

Appendix A: Knowledge Graph Embedding (KGE)

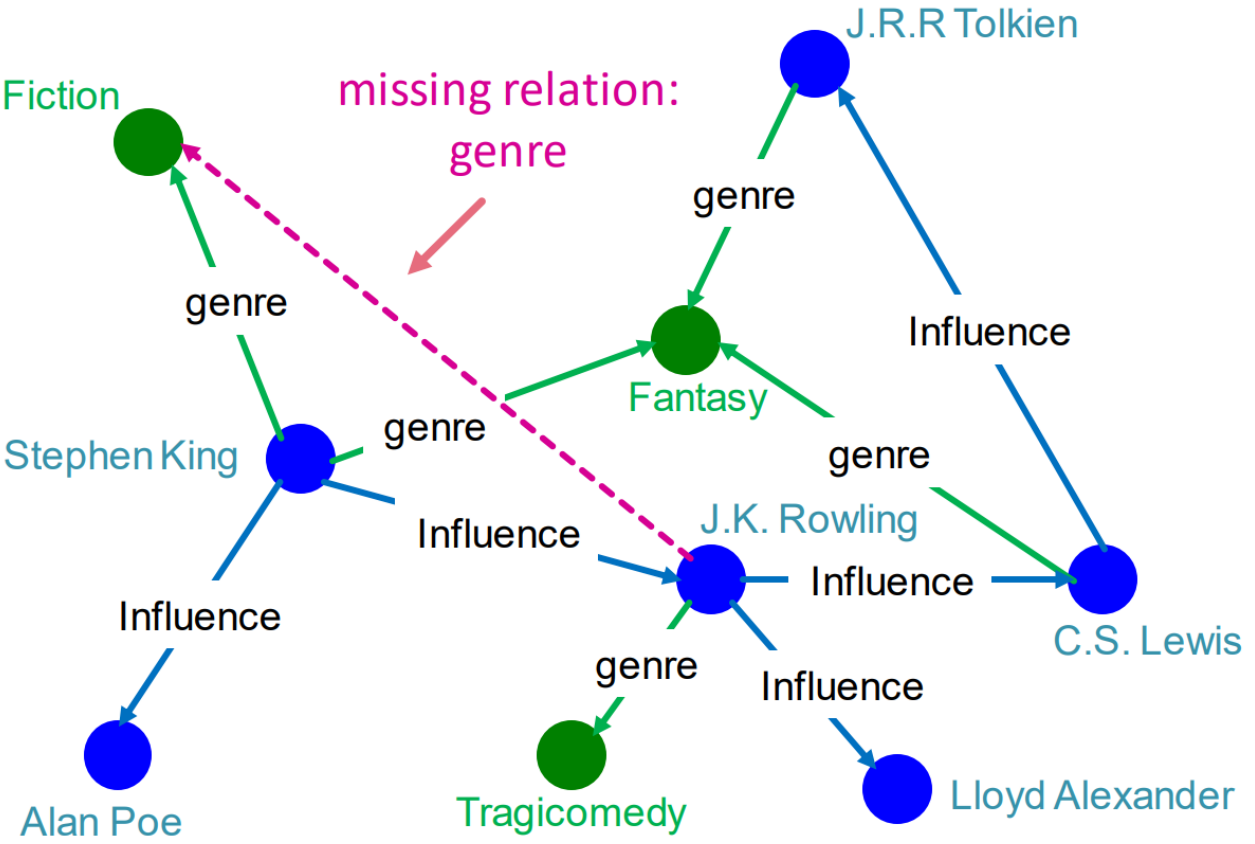
Slides adapted from: Jure Leskovec, Stanford, CS224W (2023): Machine Learning with Graphs, http://cs224w.stanford.edu

KG completion task

Given an enormous KG, can we complete the KG?

- For a given (**head**, **relation**), we predict missing **tails**.
  - (Note this is slightly different from link prediction task)

Example task: predict the tail "Science Fiction" for ("J.K. Rowling", "genre")



KG embedding (or KG representation)

- Edges in KG are represented as **triples** (h, r, t)
  - head (h) has relation (r) with tail (t)
- **Key Idea:**
  - Model entities and relations in the embedding/vector space $\mathbb{R}^d$.
    - Associate entities and relations with **shallow embeddings**
    - **Note we do not learn a GNN here!**
  - Given a true triple (h, r, t), the goal is that the embedding of (h, r) **should be close** to the embedding of t.
    - How to embed (h, r)?
    - How to define closeness?

Different models for KG embedding

We are going to learn about different KG embedding models (shallow/transductive embs):

- Different models are…
  - …based on different geometric intuitions
  - …capture different types of relations (have different expressivity)

Model	Score	Embedding	Sym.	Antisym.	Inv.	Compos.	1-to-N
TransE	$-\|\|\mathbf{h}+\mathbf{r}-\mathbf{t}\|\|$	$\mathbf{h},\mathbf{t},\mathbf{r}\in\mathbb{R}^k$	✗	✓	✓	✓	✗
TransR	$-\|\|\boldsymbol{M}_r\mathbf{h}+\mathbf{r}-\boldsymbol{M}_r\mathbf{t}\|\|$	$\mathbf{h},\mathbf{t},\mathbf{r}\in\mathbb{R}^k$, $\boldsymbol{M}_r\in\mathbb{R}^{d\times k}$	✓	✓	✓	✓	✓
DistMult	$<\mathbf{h},\mathbf{r},\mathbf{t}>$	$\mathbf{h},\mathbf{t},\mathbf{r}\in\mathbb{R}^k$	✓	✗	✗	✗	✓
ComplEx	$\mathrm{Re}(<\mathbf{h},\mathbf{r},\bar{\mathbf{t}}>)$	$\mathbf{h},\mathbf{t},\mathbf{r}\in\mathbb{C}^k$	✓	✓	✓	✗	✓

TransE

- **Translation Intuition**:

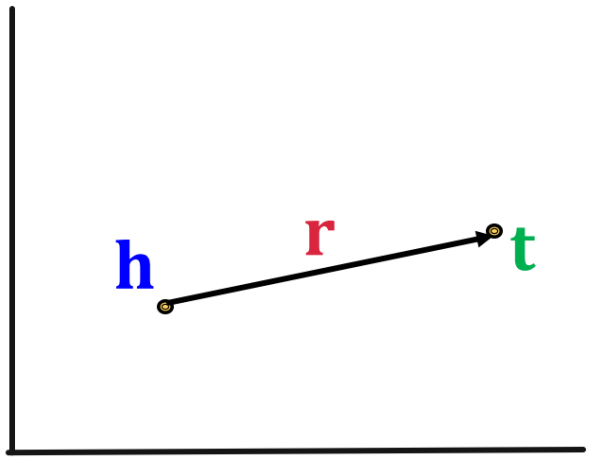
  For a triple (h, r, t), $\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^d$,

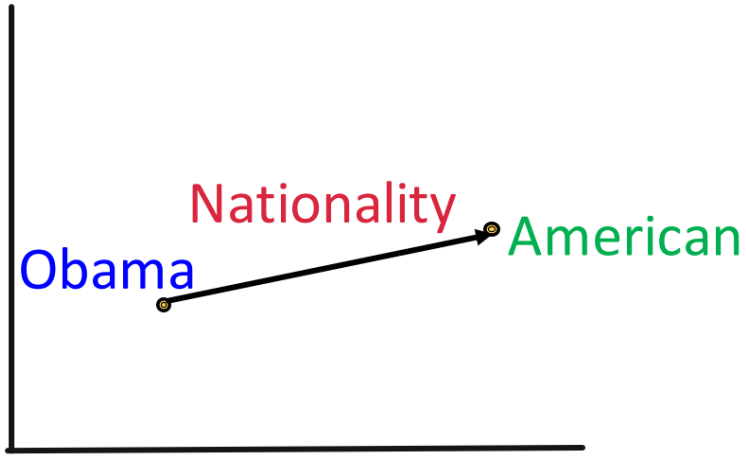
  $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ if the given fact is true

  else $\mathbf{h} + \mathbf{r} \neq \mathbf{t}$

embedding vectors will appear in boldface

Scoring function: $f_r(h, t) = -||\mathbf{h} + \mathbf{r} - \mathbf{t}||$





TransE: contrastive/triplet loss

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L, margin γ, embeddings dim. k.

1: **initialize** $\boldsymbol{\ell} \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $\ell \in L$
2: $\boldsymbol{\ell} \leftarrow \boldsymbol{\ell} / \|\boldsymbol{\ell}\|$ for each $\ell \in L$
3: $\mathbf{e} \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$
4: **loop**
5: $\mathbf{e} \leftarrow \mathbf{e} / \|\mathbf{e}\|$ for each entity $e \in E$
6: $S_{batch} \leftarrow$ sample(S, b) // sample a minibatch of size b
7: $T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets
8: **for** $(h, \ell, t) \in S_{batch}$ **do**
9: $(h', \ell, t') \leftarrow$ sample$(S'_{(h,\ell,t)})$ // sample a corrupted triplet
10: $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$
11: **end for**
12: Update embeddings w.r.t. $\sum_{((h,\ell,t),(h',\ell,t')) \in T_{batch}} \nabla [\gamma + d(\boldsymbol{h} + \boldsymbol{\ell}, \boldsymbol{t}) - d(\boldsymbol{h'} + \boldsymbol{\ell}, \boldsymbol{t'})]_+$
13: **end loop**

Entities and relations are initialized uniformly, and normalized

Negative sampling with triplet that does not appear in the KG

d represents distance (negative of score)

positive sample

negative sample

Contrastive loss: favors lower distance (or higher score) for valid triplets, high distance (or lower score) for corrupted ones

Connectivity patterns in KG

- **Relations in a heterogeneous KG have different properties:**
  - Example:
    - **Symmetry:** If the edge $(h, \text{"Roommate"}, t)$ exists in KG, then the edge $(t, \text{"Roommate"}, h)$ should also exist.
    - **Inverse relation:** If the edge $(h, \text{"Advisor"}, t)$ exists in KG, then the edge $(t, \text{"Advisee"}, h)$ should also exist.
- Can we **categorize** these relation patterns?
- **Are KG embedding methods (e.g., TransE) expressive enough to model these patterns?**

Four relation patterns

- **Symmetric (Antisymmetric) Relations:**

$$r(h,t) \Rightarrow r(t,h) \quad (r(h,t) \Rightarrow \neg r(t,h)) \quad \forall h,t$$

  - **Example**:
    - Symmetric: Family, Roommate
    - Antisymmetric: Hypernym
- **Inverse Relations:**

$$r_2(h,t) \Rightarrow r_1(t,h)$$

  - **Example** : (Advisor, Advisee)
- **Composition (Transitive) Relations:**

$$r_1(x,y) \wedge r_2(y,z) \Rightarrow r_3(x,z) \quad \forall x,y,z$$

  - **Example**: My mother's husband is my father.
- **1-to-N relations:**

$$r(h,t_1), r(h,t_2), \ldots, r(h,t_n) \text{ are all True.}$$

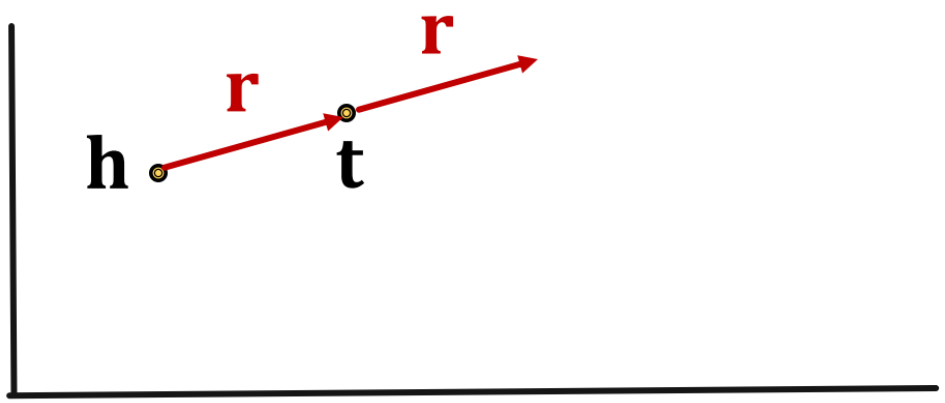
  - **Example**: r is "StudentsOf"

Antisymmetric relations in TransE

- **Antisymmetric Relations:**

$$r(h,t) \Rightarrow \neg r(t,h) \quad \forall h,t$$

  - **Example**: Hypernym
- **TransE** can model antisymmetric relations ✓
  - $\mathbf{h} + \mathbf{r} = \mathbf{t}$, but $\mathbf{t} + \mathbf{r} \neq \mathbf{h}$

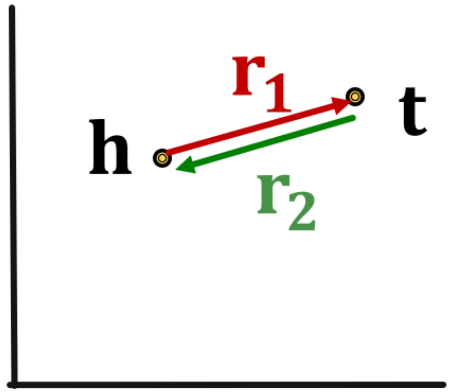


Inverse relations in TransE

- **Inverse** Relations:

$$r_2(h,t) \Rightarrow r_1(t,h)$$

  - **Example** : (Advisor, Advisee)
- **TransE** can model inverse relations ✓
  - $\mathbf{h} + \mathbf{r}_2 = \mathbf{t}$, we can set $\mathbf{r}_1 = -\mathbf{r}_2$



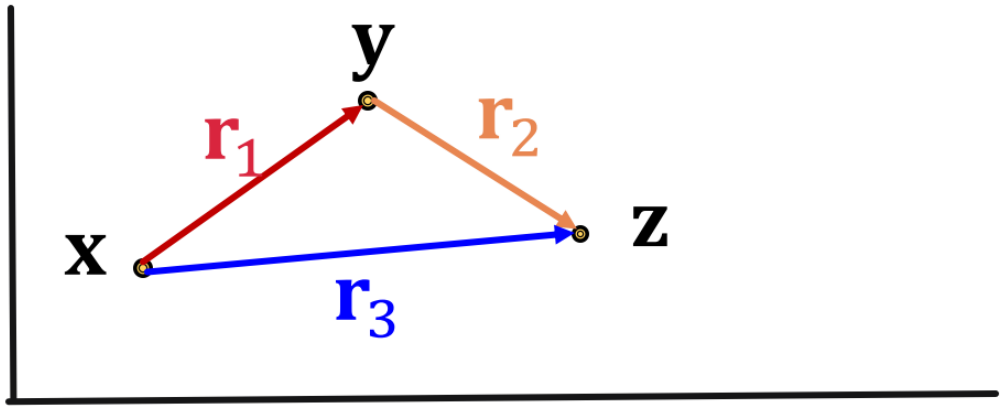
Composition in TransE

- **Composition (Transitive) Relations:**

$$r_1(x,y) \wedge r_2(y,z) \Rightarrow r_3(x,z) \quad \forall x,y,z$$

  - **Example**: My mother's husband is my father.
- **TransE** can model composition relations ✓

$$\mathbf{r}_3 = \mathbf{r}_1 + \mathbf{r}_2$$



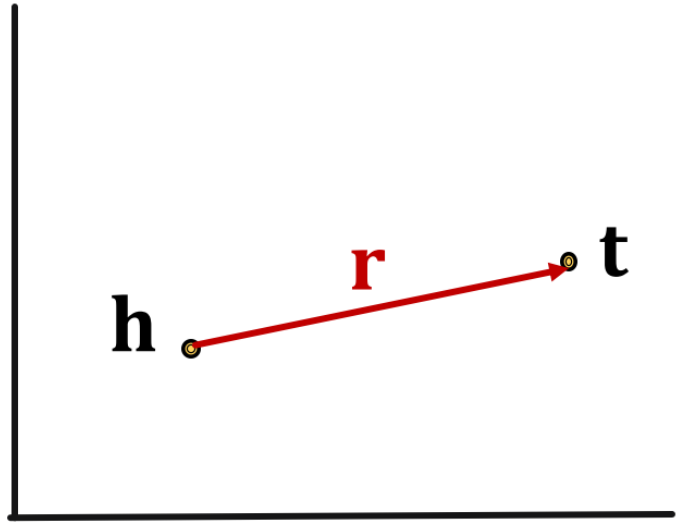
Limitation: Symmetric relations

- **Symmetric Relations:**

$$r(h,t) \Rightarrow r(t,h) \quad \forall h,t$$

  - **Example**: Family, Roommate
- **TransE cannot** model symmetric relations ✘

only if $\mathbf{r} = 0,\ \mathbf{h} = \mathbf{t}$

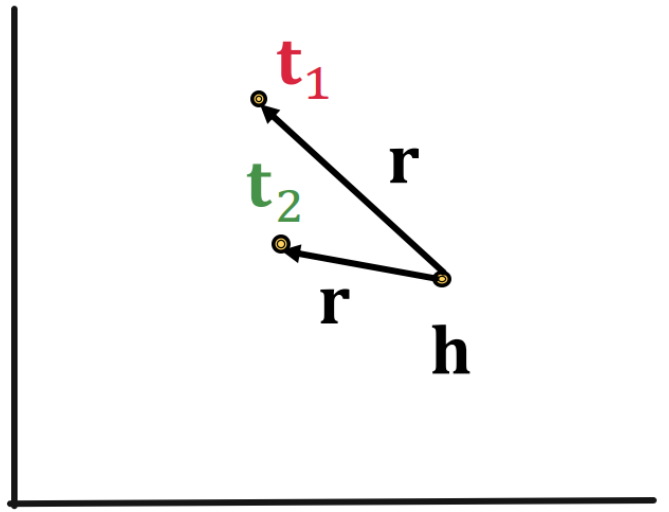


For all h,t that satisfy $r(h,t)$, $r(t,h)$ is also True, which means $\|\mathbf{h}+\mathbf{r}-\mathbf{t}\| = 0$ and $\|\mathbf{t}+\mathbf{r}-\mathbf{h}\| = 0$. Then $\mathbf{r} = 0$ and $\mathbf{h} = \mathbf{t}$, however h and t are two different entities and should be mapped to different locations.

Limitation: 1-to-N relations

- **1-to-N Relations:**
  - **Example**: (h, r, t_1) and (h, r, t_2) both exist in the knowledge graph, e.g., r is "StudentsOf"
- **TransE cannot** model 1-to-N relations ✘
  - $\mathbf{t}_1$ and $\mathbf{t}_2$ will map to the same vector, although they are different entities

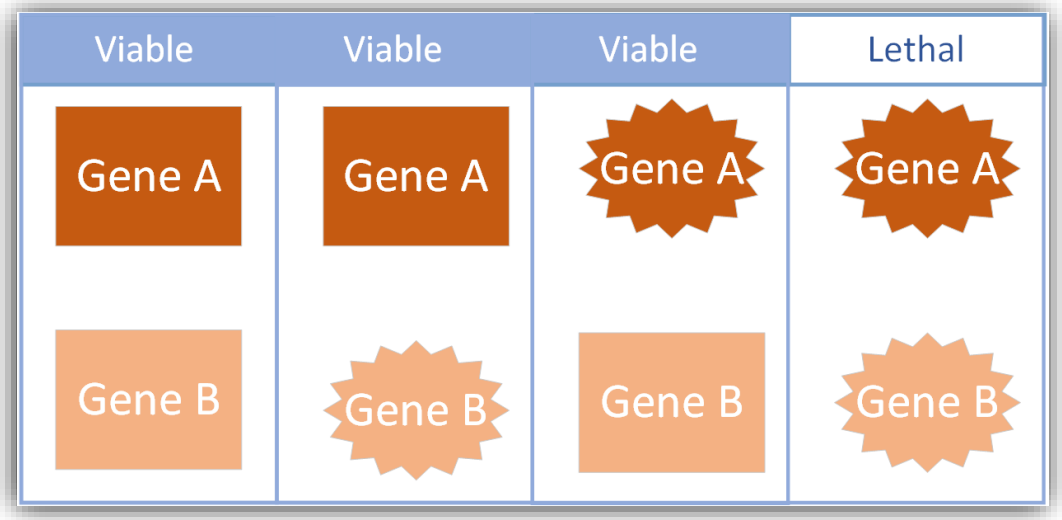
- $\mathbf{t}_1 = \mathbf{h} + \mathbf{r} = \mathbf{t}_2$
- $\mathbf{t}_1 \neq \mathbf{t}_2$ contradictory!



Appendix B: Applications in Predicting Synthetic Lethality

Synthetic Lethality (SL)

- A pair of genes is called **synthetic lethality (SL)** if:
  - The defect of a **single** gene will not affect the cell viability, and
  - The defects of **both genes** will cause cell death or significant impairment of cell fitness
- SL is a gold mine of anti-cancer drug targets:
  - Many cancers are caused by gene mutations (e.g. oncogenes)
  - Knock down SL partner of a gene with **cancer-specific mutation** will kill the cancer cells, but spare normal cells



Existing methods

- **Statistical inference** (e.g. DAISY[1])
  - 3 independent procedures (SCNA, shRNA, gene expression)
- **Supervised machine learning**
  - SVM, Random Forests, or some other ensemble methods
- **Matrix Factorization (MF)**[2]
  - Factorize the adjacency matrix down to latent representation
- **Graph Neural Network**
  - GCN, GraphSAGE, DDGCN[3]

[1] Ryan et al. DAISY: Picking Synthetic Lethals from Cancer Genomes, Cell 2014

[2] Liu et al. SL^{2}MF: Predicting Synthetic Lethality in Human Cancers via Logistic Matrix Factorization, IEEE/ACM TCBB 2020

[3] Cai et al. Dual-dropout graph convolutional network for predicting synthetic lethality in human cancers, Bioinformatics 2020

Rationale of KG4SL

- Previous methods view SLs as **independent** samples, not taking their correlation into considerations
  - Different SL pairs share mechanisms, e.g. protein-protein interaction, gene-gene co-expression, DNA damage repair
- We can use **knowledge graph** to capture the correlation

- **KG4SL**: **K**nowledge **G**raph Neural Network **for** **S**ynthetic **L**ethality Prediction

- Problem formulation: **Link prediction** between genes

Data Collection

- SL Data: A matrix representing genes interactions
- SynLethKG: A comprehensive knowledge graph of SL

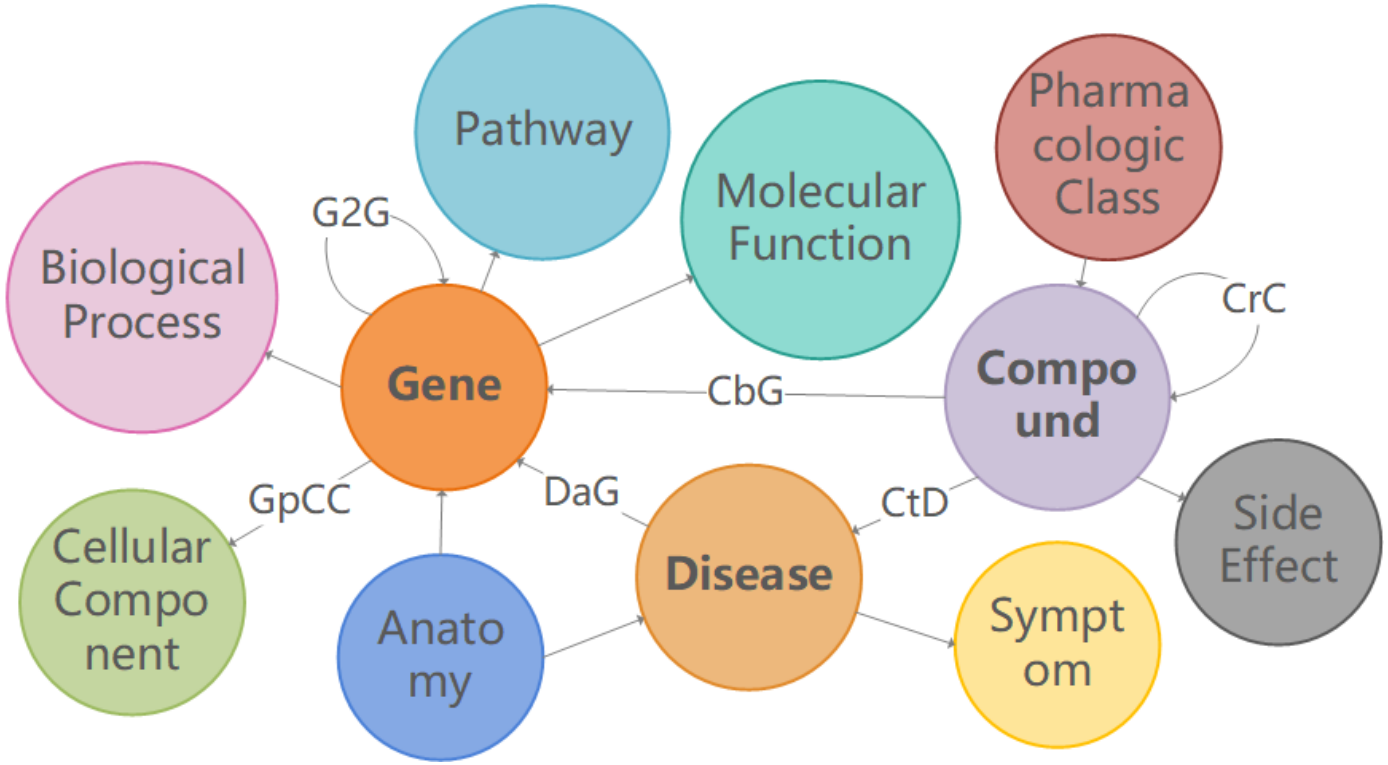
	Genes	10,004
	Interactions	72,804
SL Data	Density	0.07%
	Entity types	11
	Relationship Types	24
SynLethKG	Nodes	54,012
	Edges	2,231,921

Jie Wang, et al. "SynLethDB 2.0: A web-based knowledge graph database on synthetic lethality for novel anticancer drug discovery", Database: the journal of biological databases and curation, Vol. 2022, baac030, 2022.

SynLethKG

SynLethKG is a knowledge graph about SL gene pairs

- 11 types of biomedical entities
- 27 types of relationships
- Representing features and relationships between genes, cancers and drugs



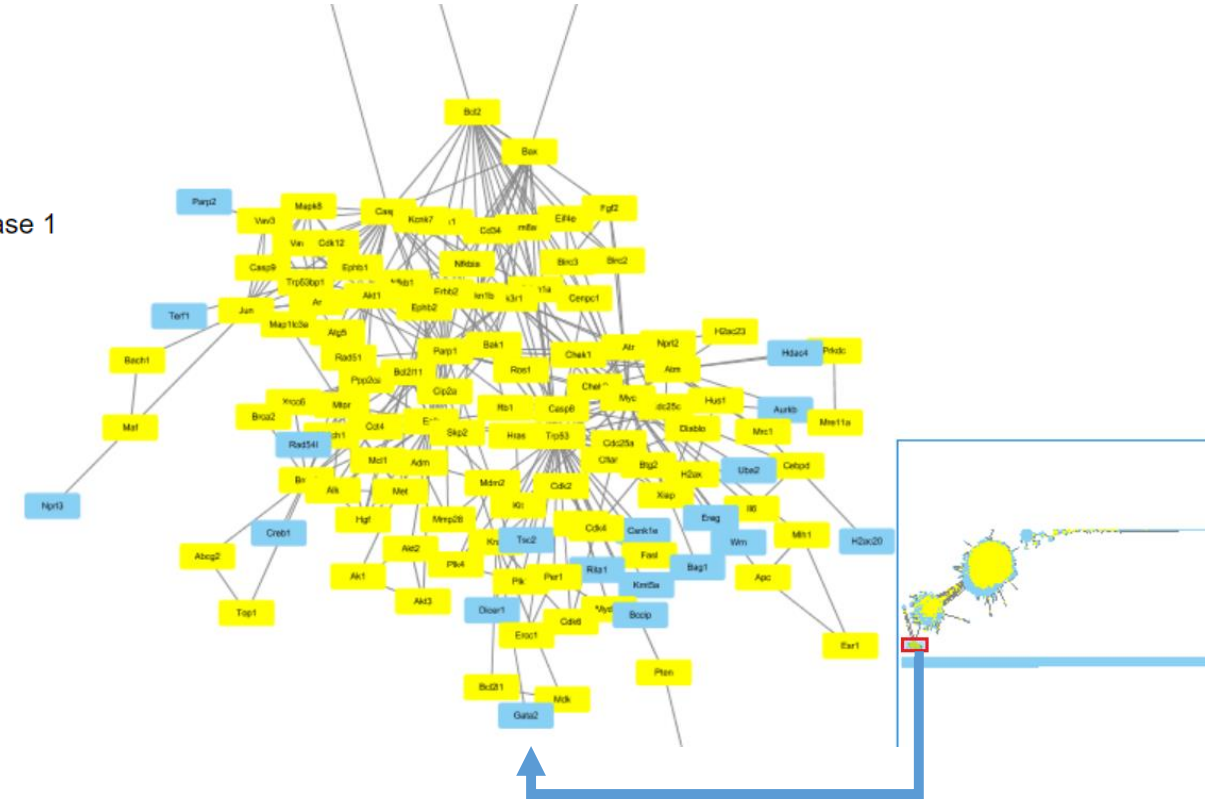
SynLethKG

- SynLethKG is an attribute graph, i.e. nodes and edges in the graph have properties
  - Example:
    - license: CC0 1.0
    - identifier: 142
    - Organism: Homo sapiens
    - chromosome: 1
    - name: PARP1
    - description: poly(ADP-ribose) polymerase 1
    - source: Entrez Gene
    - url: http://identifiers.org/ncbigene/142

- If we do not use the attributes, then the graph can be represented by triplets in the format of

  <head, relationship, tail>

Visualization by Cytoscape



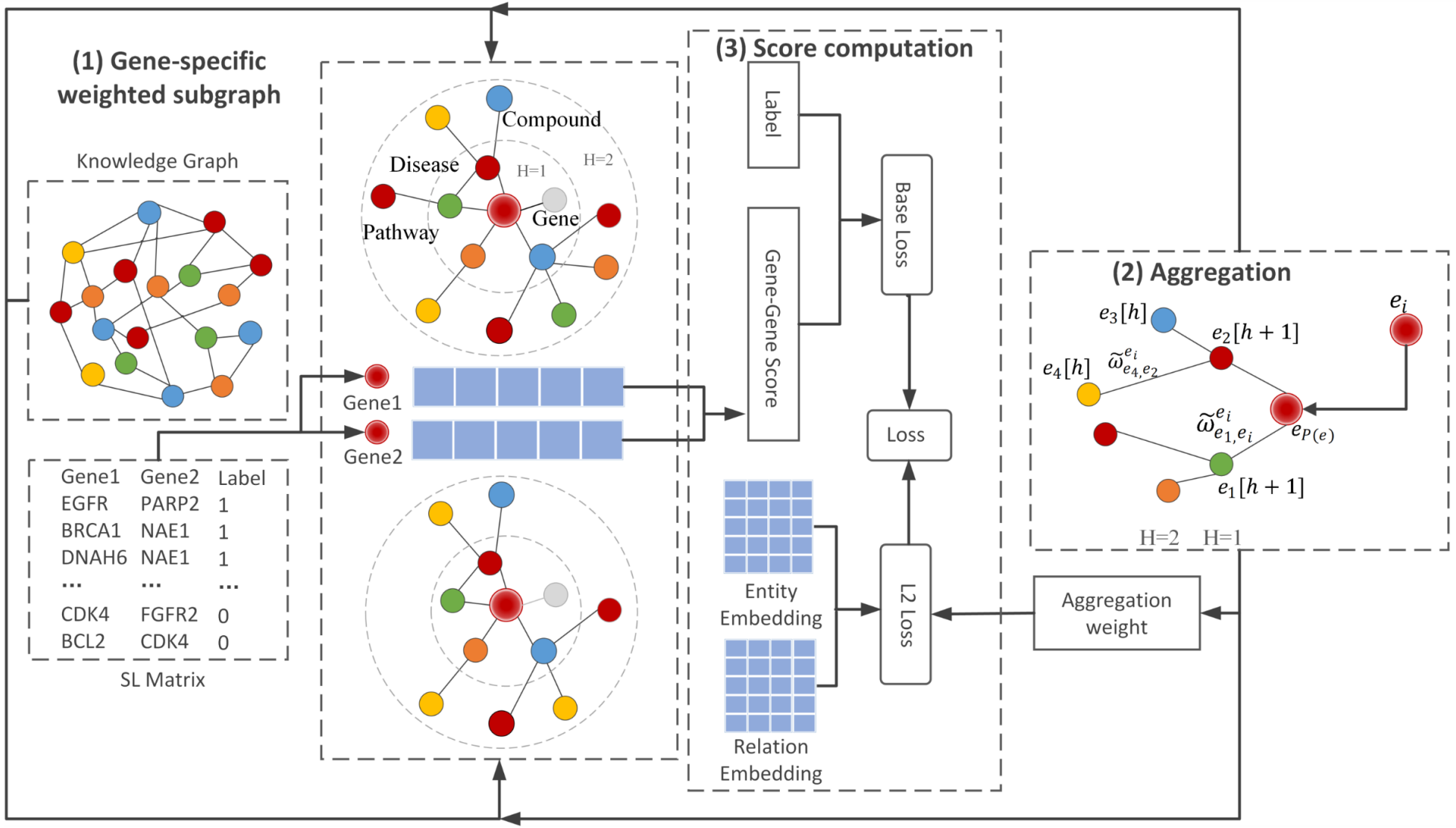
Sources of knowledge in SynLethKG

上海科技大学 ShanghaiTech University

Types of entity	# Nodes
[CellularComponent]	1670
[Gene]	67062
[BiologicalProcess]	12703
[SideEffect]	5726
[MolecularFunction]	3203
[Pathway]	2069
[Disease]	137
[Compound]	2595
[PharmacologicClass]	377
[Anatomy]	402
[Symptom]	453

Types of relationship	# Edges	Sources
(Anatomy, downregulates, Gene)	31	Bgee
(Anatomy, expresses, Gene)	617175	Bgee; TISSUES
(Anatomy, upregulates, Gene)	26	Bgee
(Compound, binds, Gene)	16323	Drugbank; BindingDB
(Compound, causes, Side Effect)	139428	SIDER 4.1
(Compound, downregulates, Gene)	21526	LINCS L1000
(Compound, palliates, Disease)	384	PharmacotherapyDB
(Compound, resembles, Compound)	6266	Dice similarity of ECFPs
(Compound, treats, Disease)	752	pharmacotherapyDB
(Compound, upregulates, Gene)	19200	LINCS L1000
(Disease, associates, Gene)	24328	DISEASES; DisGeNET; DOAF
(Disease, downregulates, Gene)	7616	STARGEO
(Disease, localizes, Anatomy)	3373	Medline
(Disease, presents, Symptom)	3401	Medline
(Disease, resembles, Disease)	404	Medline
(Disease, upregulates, Gene)	7730	STARGEO
(Gene, covaries, Gene)	62987	ERC
(Gene, interacts, Gene)	148379	PPI
(Gene, participates, Biological Process)	619712	NCBI gene2go
(Gene, participates, Cellular Component)	97652	NCBI gene2go
(Gene, participates, Molecular Function)	110042	NCBI gene2go
(Gene, participates, Pathway)	57441	WikiPayhways; Reactome; Pathway Commons
(Gene, regulates, Gene)	267791	LINCS L1000
(Pharmacologic Class, includes, Compound)	1205	DrugCentral

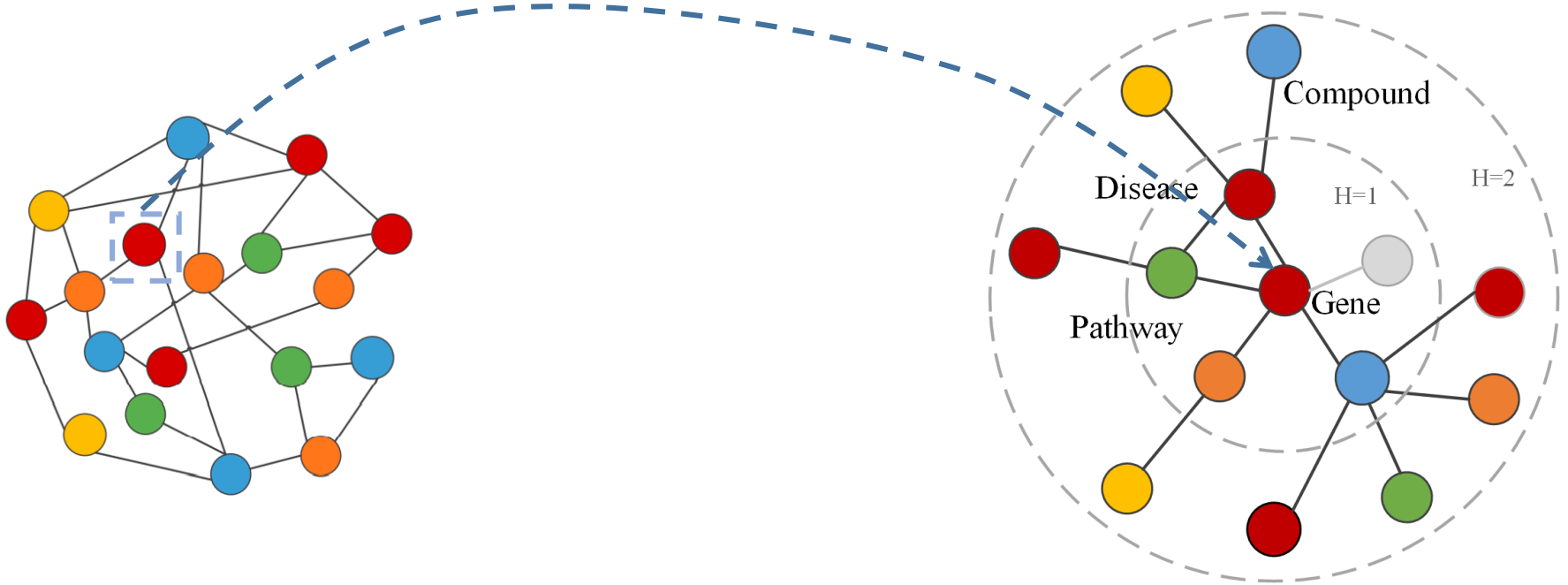
KG4SL overview



Shike Wang#, Fan Xu# (joint first authors), et al. "KG4SL: Knowledge graph neural network for synthetic lethality prediction in human cancers", Bioinformatics, 37(Suppl_1):i418-i425 (ISMB/ECCB 2021 special issue), 2021.

Gene-specific weighted subgraph

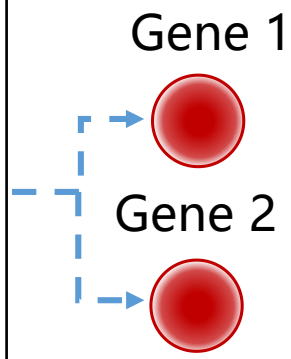
- **Zoom into one gene subgraph in KG**:
  - Examine H-order graph neighborhoods
  - Sample a fixed-size neighborhood for each node
  - Compute the weighted average combination of neighborhoods

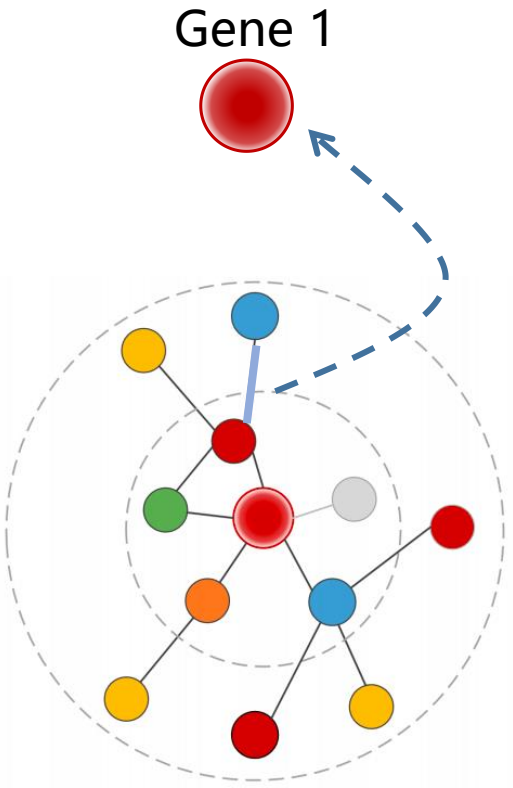


Aggregation

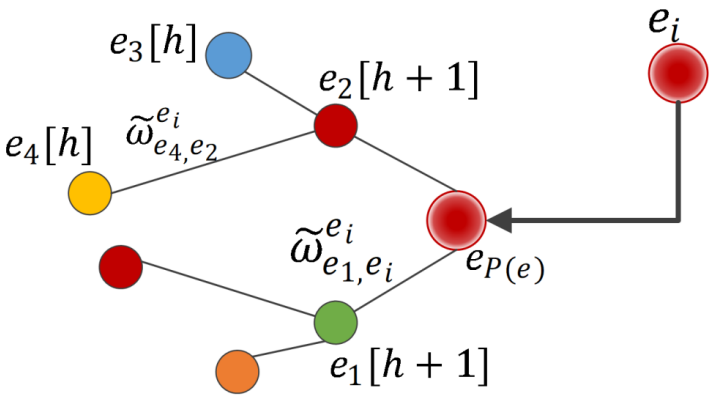
- **Learn a gene feature:**
  - Contribute different importance to the interactive gene
  - Aggregate the entity representation e, and its neighborhood into a single vector

Gene1	Gene2	Label
EGFR	PARP2	1
BRCA1	NAE1	1
DNAH6	NAE1	1
…	…	…
CDK4	FGFR2	0
BCL2	CDK4	0





Gene 2 subgraph

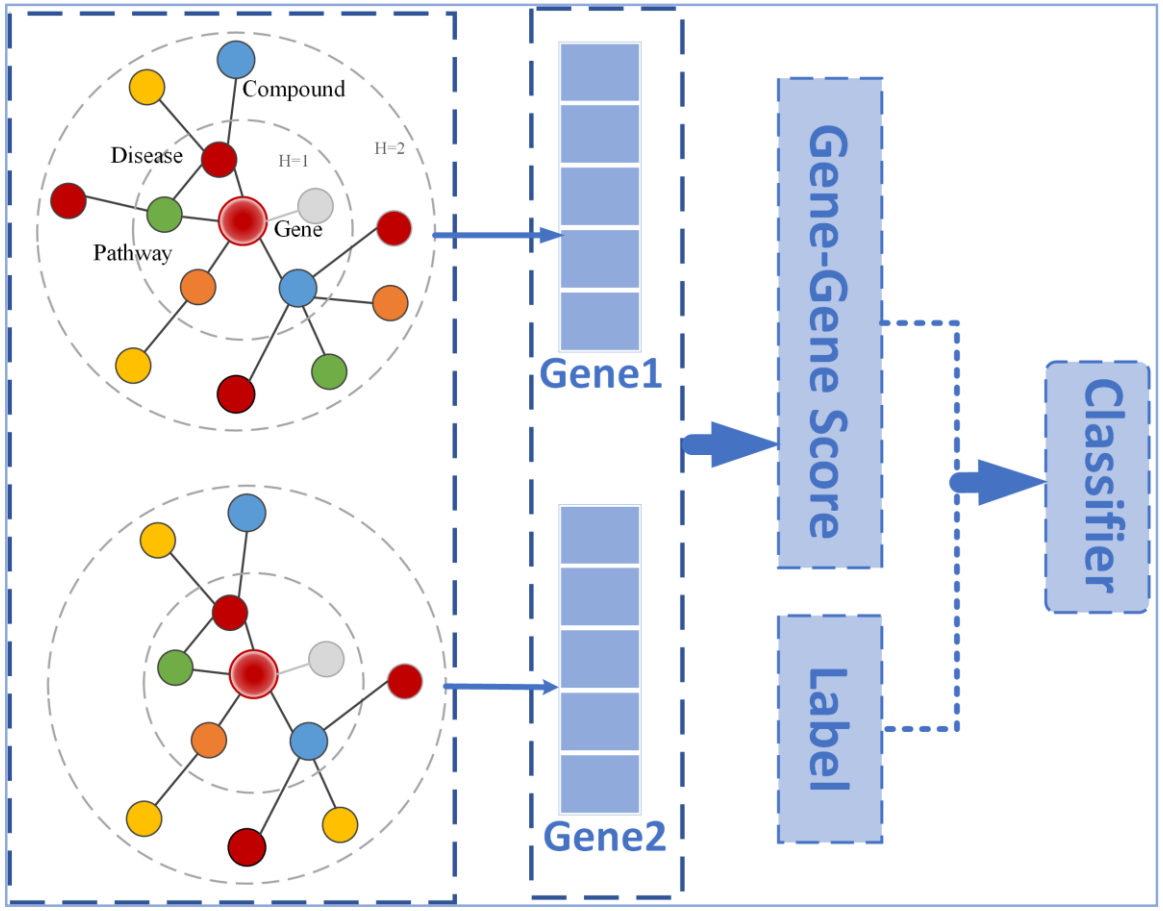


Message passing

Computation of SL score

- **Dot product predictor:**
  - Gene-relation scoring
  - Gene-gene scoring

- **Loss:**
  - Cross entropy loss
  - L2 Regularizer



Comparison with baselines

	MF			RW			GNN				KG
	SL^2MF	GRSMF	HOPE	Deep Walk	node2 vec	LINE	GCN	GraphS AGE	GAT	DDGC N	KG4SL
AUC	0.7811	0.9184	0.7776	0.8451	0.8362	0.8233	0.8329	0.8398	0.7914	0.8491	0.9470
AUPR	0.8635	0.9362	0.7410	0.8600	0.8523	0.8327	0.8727	0.8775	0.8462	0.8998	0.9564
F1	0.7446	0.8339	0.7089	0.7562	0.7503	0.7380	0.8508	0.8569	0.8152	0.8154	0.8877

Baselines:

- Matrix factorization (MF)
- Random walk (RW)
- Graph neural network (GNN)

Impact of Knowledge Graph

	KG	SL	KG+SL	
	TransE	GCN	TransE +GCN	KG4SL
AUC	0.5870	0.8329	0.9063	0.9470
AUPR	0.6100	0.8727	0.9138	0.9564